

Takneek 2026 · Mid Prep · Technical Report

The Edge of Magic

A Standalone TinyML Spell-Recognition Wand
ESP32-WROOM-32 · BNO055 sensor fusion · 50 cm shaft

Pipeline BNO055 IMU fusion → quaternion-anchored frame → int8 CNN → five-stage rejection cascade

Inference entirely on-chip, TensorFlow Lite for Microcontrollers

Tensor 64 steps × 12 channels · translation and rotation decoupled

Model 7,174 parameters · 22,520-byte int8 flatbuffer · 6 classes

Wand 500 mm tube · sensor 130 mm below the tip · +X along the shaft

Serial offloading not compiled into the demo binary

Document revision R4 · 1 September 2026

Contents

1	Executive summary and status
2	Requirement analysis: every rule mapped to a design response
3	Hardware architecture, bill of materials, and the physical stack-up
4	Spell vocabulary, kinematic definitions, and the lever-arm model
5	Data acquisition: sampling, segmentation, collection protocol
6	The signal processing chain, step by step
7	The TinyML model: architecture, training, quantisation
8	False-positive rejection — the Muggle Movement defence
9	Validation methodology, the synthetic quarantine, and the corpus audit
10	Firmware implementation walkthrough
11	Latency and memory budget
12	On-device processing compliance
13	Execution plan for the remaining build window
14	Risk register and fallbacks
15	Appendix: repository layout and reproduction

1 Executive summary and status

This report describes a self-contained wearable wand that recognises five spell gestures and, equally importantly, refuses to recognise anything else. All sensing, signal conditioning, neural inference and decision-making run on a single ESP32-WROOM-32. Nothing is offloaded; the demonstration firmware does not contain a call to `Serial.begin()` at all.

Motion is captured by a **BNO055** running in IMU fusion mode, which does its own sensor fusion on an onboard Cortex-M0 and returns gravity-free linear acceleration, bias-compensated angular rate, and an orientation quaternion at 100 Hz. That quaternion is what the rest of the design is built around: every gesture is rotated into the frame the wand occupied at the instant the caster began, which *decouples translation from rotation* instead of leaving them entangled the way a raw body-frame representation does. The resulting 64×12 tensor is classified by a 7,174-parameter depthwise-separable 1-D CNN, quantised to int8, into four compulsory spells, one bonus spell, and an explicitly trained *NOISE* class. A five-stage rejection cascade wraps the network and decides whether to fire at all.

The wand is a 500 mm tube. The ESP32 and the power system live at the butt, where the hand grips; the OLED faces the caster from mid-shaft; the BNO055 is mounted **130 mm below the tip** with its +X axis running along the shaft towards the tip; the eight-LED ring is at the tip itself. That geometry is not packaging trivia and it is not confined to §3 — it is an input to the signal model. A sensor bolted 370 mm from the caster's hand does not measure the hand; it measures the hand *plus a lever arm*, and for every gesture in this vocabulary the lever term is three to ten times the larger. R4 exists because the previous revision's bootstrap generator did not know that.

1.1 Honest statement of what has and has not been measured

At the time of writing, no gestures have been recorded from human casters. This report therefore contains **no accuracy figure derived from real data**, and deliberately does not invent one. What it does contain is a fully built, executed and verified pipeline, plus **ten** defects that were found and fixed before any data existed — five of them in R4, and four of

those five found *because* the wand's real geometry was written into the model. The table separates the two categories, because a report that blurs them is worthless.

Item	Status	Evidence
Python ↔ C++ preprocessing parity	Measured	Worst per-element difference 6.32×10^{-6} across 60 captures, against a 10^{-3} tolerance. Stage 1 <i>verdict</i> parity now checked too, after a negative control walked through the old harness untouched. Three negative controls, all caught (§9.2).
Firmware compiles, all six translation units	Measured	<code>tools/hostcheck/check.sh</code> type-checks every firmware source on a laptop against stubbed Arduino and TFLM headers. Not a functional test; it removes the flash-cycle-per-typo loop (§10.5).
Constant and channel-layout parity	Measured	10 pytest assertions pass; 1 skipped by design until thresholds are calibrated on real data.
Model size and parameter count	Measured	7,174 parameters; 22,520-byte int8 <code>.tflite</code> ; int8 input and output.
Cost of int8 quantisation	Measured	<i>Zero</i> . int8 argmax accuracy is identical to float. Worst channel still resolves to 58 int8 levels (§7.5).
Threshold calibration effect	Measured	Cascade accuracy $0.452 \rightarrow 0.983$ on the same quantised model, purely from calibrating three thresholds instead of guessing them (§8.4).
Accelerometer reportable range	Measured	Per-axis linear acceleration reaches exactly 5.00 g — the ± 4 g rail <i>plus</i> the gravity the fusion subtracts back out. R3's divisor of 3.0 was derived with the sign the wrong way round and railed 2.4% of all samples (§6.5).
Stage 1 gate operating point	Measured on <i>bootstrap</i>	98–100% of spell windows pass; 61% of noise windows stopped before inference (50% impact, 11% no motion). Both arms swept jointly, not guessed (§8.3).
Bootstrap corpus physical plausibility	Measured on <i>bootstrap</i>	All six classes inside their bands for peak rate, peak acceleration, tip speed and duration; lever-arm coupling $\rho = 0.63\text{--}0.93$ within each rotational class (§9.5). The same script is the acceptance test for the real corpus.
Inference latency on ESP32	Estimated	~66k MAC operations; 2–5 ms expected. The firmware measures and prints the real value.
Classification accuracy	Not measured	Requires real captures. §13 gives the collection plan; §9.1 gives the protocol that will produce the number.

WHY THIS SECTION EXISTS

An earlier iteration of this project produced a leave-one-caster-out accuracy of 100.0% with a standard deviation of 0.00% across six folds. That number came from a generator, not from people, and it very nearly reached a slide. Real six-axis data from real humans does not behave that way. The synthetic quarantine described in §9.4 is the mechanism that makes a repeat impossible, and this table is its documentation.

1.2 What changed in revision R3, and why it is not a drop-in swap

R2 of this design used an MPU-6050 and removed gravity in software. R3 uses a BNO055. The wiring change is trivial; the consequences are not, and three of them are load-bearing.

Area	R2 (MPU-6050)	R3 (BNO055, IMU fusion)
Gyro bias	Mean of the pre-roll window, in software	Estimated continuously by the fusion. Software step deleted ; replaced by a calibration-status arming gate.
Orientation	Complementary filter, roll and pitch only	Full quaternion from the chip. Filter deleted — and with it the step our own negative control proved was the easiest thing here to get wrong.
Gravity	Projected and subtracted in software	Already removed by the chip. Step deleted .
Feature frame	Sensor body frame — translation smeared through the wand's own rotation	Anchored to gesture start — translation and rotation separated (§6.3)
Tensor	64×8	64×12 , four of them the relative quaternion
Accel range	± 8 g, chosen for Stupefy headroom	Locked to ± 4 g by the fusion, on the <i>raw</i> signal. R4 corrected what that implies: linear acceleration is reportable to ± 5 g, and sharp casts saturate (§6.5).
Impact rejection	Peak magnitude above 7.5 g	Unreachable at ± 4 g. Replaced by a <i>spike-width</i> test (§8.3).
Free-fall detection	$ a $ near zero for 50 ms	Not visible in fusion mode. Stage removed , and covered instead by the trigger and the impact test (§8.2).

The counsel we were originally given was to disable the fusion (AMG mode) so that the existing software chain would not subtract gravity twice. That reasoning identifies a real hazard and draws the wrong conclusion: the fix is to delete the software chain, not to switch off the most valuable thing the sensor does. Running a hand-written complementary filter on a chip that contains a Kalman-class estimator is paying for a part and then declining to use it.

1.3 What changed in revision R4: the wand became part of the model

R3 was correct about the sensor and silent about the object it is bolted to. The bootstrap generator that de-risked the whole pipeline described each spell as *two independently hand-tuned curves*: an angular rate programme, and, separately, an acceleration programme. Nothing connected them.

That is not a small liberty. It is the wrong physics, and it is wrong in the direction that flatters the model. On the real wand the accelerometer channel is very nearly a deterministic function of the gyro channel, because the sensor is 370 mm up a rigid shaft from a hand that is 80–330 mm behind it. A generator that treats acceleration as free hands the network a channel carrying independent information that the hardware will never provide, and the network learns a decision rule that does not exist.

R4 replaces those curves with rigid-body kinematics driven by the measured geometry (§4.2). The caster's motion is now specified as the two things a human actually commands — how the wand rotates, and where the hand goes — and everything the accelerometer sees is derived. Four consequences followed immediately, and none of them were anticipated:

Consequence	What it changed
The accelerometer divisor was wrong	<code>NORM_ACC_LIN</code> was 3.0 on the reasoning that "gravity consumes 1 g of the 4". Gravity is <i>subtracted</i> , so it extends the range rather than eating it: the true bound is 5 g and the corpus reaches exactly that. At 3.0 the tensor railed on 2.4% of all samples, including the peak of every sharp flick — the part that identifies the gesture (§6.5).
The Stage 1 energy gate was strictly dominated	Swept jointly rather than per-feature, R3's 2.20 / 1.00 / 1.80 lost 2.0% of spells to stop 14.4% of noise. 0.90 / 0.30 / 1.00 loses <i>none</i> and still stops 11.1% (§8.3).
The impact sweep had been truncated	Its grid stopped at 2.4 g because R3 believed 3 g was the ceiling. A sweep whose grid stops short does not report that it stopped short — it reports a winner (§8.3).
An over-long trigger hold produced a classified spell	A six-second hold read 230 samples of overwritten ring buffer, spliced across the write pointer, carrying a duration of exactly <code>REJ_MAX_DURATION_S</code> — which passed the duration bound and went to the network (§10.4).

Three of the four are defects that existed in R3 and were invisible to every test R3 had. They became visible the moment the synthetic data started obeying the same physics the hardware does. That is the argument for modelling the object rather than the signal, made concretely rather than in the abstract.

R4 also consolidated the firmware. The tree had accumulated *two* incompatible BNO055 implementations — the fusion driver the whole pipeline is built on, and a second one running the chip as a plain 6-axis sensor, each defining a struct called `RawSample` with different fields. Neither was reachable from a build, so nothing failed. There is now exactly one driver, in `firmware/common/`, and all three firmware targets link it (§10.1).

2 Requirement analysis

Requirement	Design response	Where
Inference strictly on the microcontroller	TFLM int8 CNN in flash; demo build compiles no serial code at all	§10, §12
Four compulsory spells classified in real time	6-class CNN over a 64×12 tensor	§7
Bonus spells	Stupefy added as class 4, chosen to be kinematically orthogonal to the other four	§4
Muggle Movement — no false positives	Trained NOISE class plus a five-stage cascade, tuned against an asymmetric cost	§8
Directional movements explicitly defined	Body-frame convention fixed in §4; the anchored frame makes it independent of which way the caster faces	§4, §6.3
Quick-Draw latency, printed on screen	<code>LATENCY_MS</code> measured from confirmed trigger release and rendered on the OLED every cast	§11
SRAM management and quantisation	Static buffers only, no heap on the hot path; full-integer int8 with int8 I/O, measured to cost nothing	§7.5, §11.2
Wand Chooses the Wizard — invariance	Anchoring removes facing; quaternion-conjugating grip augmentation covers wrist roll; LOCO validation	§7.4, §9.1
Budget cap ₹2,000	At risk. The BNO055 alone is 55–65% of the cap; see §3.1 for the arithmetic and the levers	§3.1
Anti-overfitting strategy	LOCO, physically exact augmentation, label smoothing, dropout, held-out caster	§7.3, §7.4

3 Hardware architecture, bill of materials, and the physical stack-up

3.1 Bill of materials, and a budget problem that needs a decision

READ THIS BEFORE ANYTHING ELSE IN THIS SECTION

A BNO055 breakout in India retails between roughly ₹1,100 (7Semi, plus 18% GST) and ₹1,500 (ThinkRobotics). At those prices the sensor alone consumes 55–65% of the ₹2,000 cap, and the complete build lands between **₹2,160 and ₹2,810 — over the limit** (₹2,607 at the 7Semi price with GST). The MPU-6050 it replaced cost ₹75. This is not a rounding error and it will not resolve itself; §3.2 lists the levers and the one question that decides which of them is needed.

#	Component	Specification	Qty	₹	Purpose
1	ESP32-WROOM-32 DevKit	30-pin, CP2102	1	340	MCU, 240 MHz dual core, 520 KB SRAM
2	BNO055 breakout	9-DOF, onboard fusion, I2C	1	850–1,500	Motion sensing <i>and</i> orientation. No spare — see §14.
3	SSD1306 OLED	0.96", 128×64, I2C	1	165	Spell name, confidence, latency readout
4	WS2812B ring	8 addressable LEDs	1	135	Per-spell colour feedback
5	Tactile switch + 100 nF	6×6 mm, ceramic	1	15	Hold-to-record cast trigger with RC debounce
6	Passive piezo buzzer	<5 mA	1	20	Audible cast confirmation
7	Li-Po cell	3.7 V, 1000 mAh, JST-PH	1	280	Untethered operation
8	TP4056 module	with DW01 protection	1	40	Charging and cell protection
9	MT3608 boost converter	adjustable, set to 4.50 V	1	45	Single 4.5 V rail — see §3.4
10	Passives	470 µF, 1000 µF, 100 nF ×3, 470 Ω, 100 kΩ ×2, slide switch	—	50	Filtering, debounce, LED termination, battery sense
11	Perfboard, headers, wire	—	—	90	Assembly
12	PVC pipe 25 mm + end caps	500 mm length	1	80	Wand shaft — the length is a model input, see §3.5 and §4.2
13	Heat-shrink, tape, adhesive	—	—	50	Finishing and strain relief
Total				2,160–2,810	₹160–810 over the cap

No diodes appear in this BoM. The 1N5819 reverse-polarity guard was unobtainable, and the 1N4007 in the LED branch became unnecessary once the boost converter was set to 4.50 V instead of 5.00 V — see §3.4. The cost of dropping the 1N5819 is that reverse-polarity protection is now procedural rather than electrical: check Li-Po polarity with a multimeter before the first connection and mark the verified orientation, because JST-PH keying does not help when a vendor crimped the connector backwards.

3.2 The levers, in the order they should be pulled

1. **Ask the coordinators whether a club-borrowed or already-owned BNO055 counts against the cap.** This question was already open and is now the single most consequential one on the project. It is a five-minute conversation and it decides whether any of the following is needed.
2. **Source a generic GY-BNO055 rather than a branded breakout.** Unbranded modules list substantially below the 7Semi and ThinkRobotics prices. Same silicon; verify `CHIP_ID` reads `0xA0` on arrival, because clone modules in this family sometimes carry a different part.
3. **Drop the battery path (₹365) and run from a USB power bank** housed in the handle. This preserves the on-device-inference claim completely — a power bank carries no data lines and cannot receive an offloaded computation — but it is visually weaker and invites a question from a judge, so it is third rather than second.
4. **Drop the buzzer (₹20) and the enclosure spend (₹80)** in favour of cardboard or a wooden dowel, which the problem statement explicitly permits. Saves ₹100 and costs presentation marks; a last resort.

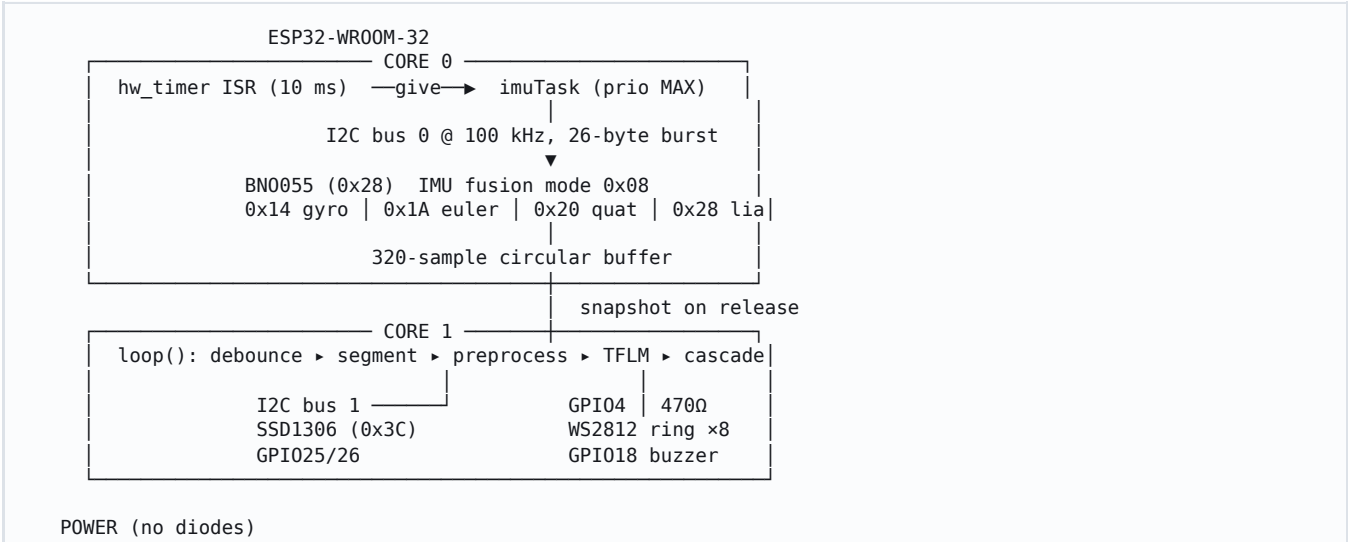
What should *not* be cut is the OLED: the problem statement requires the latency variable to be printed on screen, so it is a compliance item, not a decoration.

3.3 IMU configuration and the reasoning for each value

These register writes are part of the frozen contract with the training data. Unlike the MPU-6050, none of them is a *range* setting — in fusion mode the algorithm owns the ranges — which removes an entire class of silent error, where a scale constant and a register drift apart and every capture is quietly rescaled.

Setting	Value	Reasoning
OPR_MODE (0x3D)	0x08 IMUPLUS	Accelerometer + gyroscope fusion, magnetometer off . Not NDOF (0x0C): the magnetometer is the one sensor that is actively hostile in a demo hall — steel tables, laptops, speakers, and a WS2812 ring drawing ~150 mA a few centimetres away. Absolute heading is not needed, because §6.3 anchors every gesture to its own starting orientation and heading cancels. IMU mode also needs only <i>gyro</i> calibration, which reaches full status after a few seconds of stillness, instead of a figure-of-eight magnetometer dance in front of the judges.
UNIT_SEL (0x3B)	0x00	m/s ² , degrees per second, degrees. This is the power-on default and is written explicitly anyway, so that a chip left in mg or rad/s by an earlier experiment cannot silently rescale the entire dataset.
Bus speed	100 kHz	Not 400 kHz. The BNO055 stretches the I2C clock, and a too-fast bus with a short timeout produces an intermittent hang — it passes every quick bench test and dies mid-demo. <code>Wire.setTimeout(1000)</code> for the same reason. 100 kHz costs 2.4 ms per 26-byte burst, which is affordable on a core that does nothing else.
Accel range	±4 g (<i>locked</i>)	Not a choice; every fusion mode fixes it. The rail is on the <i>raw</i> signal, and the fusion subtracts gravity afterwards, so what the linear-acceleration registers can report is ±5 g per axis — the rail plus one gravity, not minus it. R3 had this backwards; §6.5 has the correction and what it cost. The rail is still what forced the impact-test redesign in §8.3.
Gyro range	±2000 dps (<i>locked</i>)	Also fixed by the fusion, and fortunately the value we would have chosen: a hard wrist flick exceeds 1000 dps, and at ±500 dps every flick would clip flat and become indistinguishable across the four spells that contain one.
Scale factors	100 LSB/(m/s ²), 16 LSB/dps, 16384 LSB/quaternion unit	Datasheet constants, not measurements. Note that the "lay it flat and read the vertical axis" calibration procedure used for the MPU-6050 does not work here : linear acceleration is gravity-free and reads ~0 at rest on every axis. The equivalent check is to read the gravity vector register and confirm ~981 counts on the vertical axis.

3.4 Bus, core and power architecture




```

Li-Po 3.7V ▶ TP4056+DW01 ▶ slide switch ▶ MT3608 @ 4.50 V
├─ ESP32 VIN (470µF || 100nF)
├─ ring V+ (1000µF at ring)
ESP32 onboard AMS1117 → 3V3 ──┬─ BNO055 VIN
                                └─ OLED VCC

```

Why the boost is set to 4.50 V and not 5.00 V. This single change removed both diodes from the design. A WS2812's logic threshold is approximately $0.7 \times V_{DD}$; at 5 V that is 3.5 V and the ESP32 only drives 3.3 V — marginal on a cool bench, and a source of dropped pixels once the chip warms up mid-demo. The original fix was a diode in the ring's supply. Setting the whole rail to 4.50 V puts the threshold at 3.15 V, which 3.3 V logic clears, while still leaving the AMS1117 about 1.2 V of headroom to produce 3.3 V. One trimpot turn replaces a component.

Set the MT3608 before connecting anything to its output. These ship at arbitrary settings and can arrive well above 5 V. Battery and switch on the input, output bare, multimeter on the output, turn the trimpot until it reads 4.50 V.

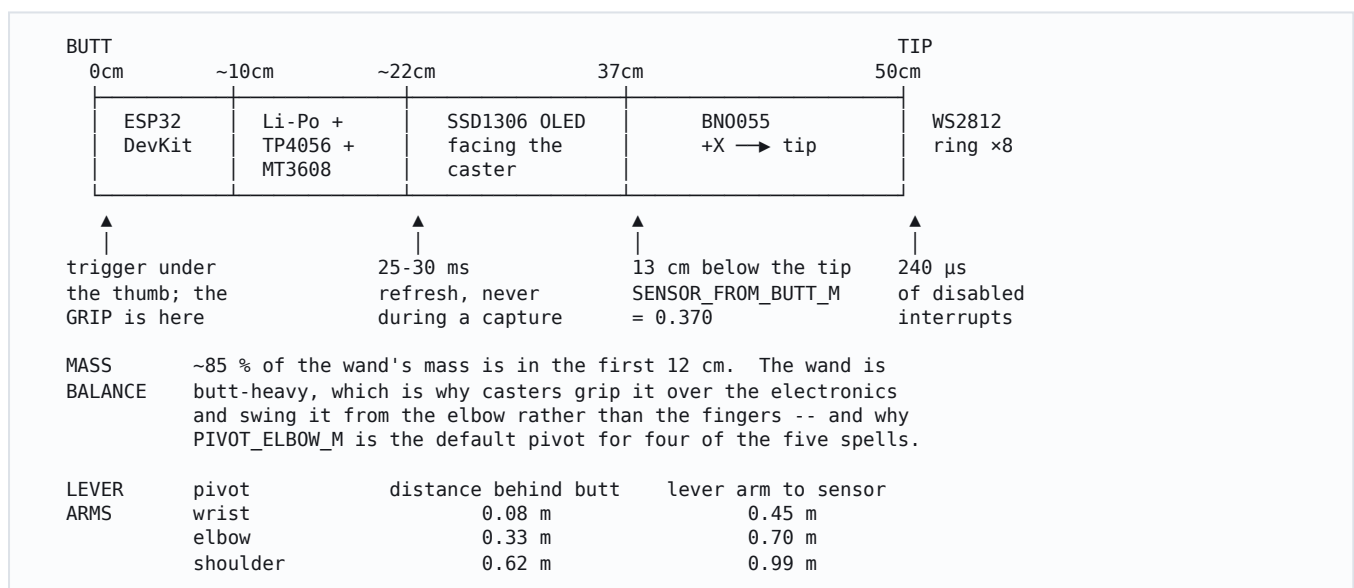
Why two I2C buses. Pushing a full 128×64 frame to the SSD1306 occupies its bus for 25–30 ms. Sharing a bus with the sensor would mean the sampler simply cannot read for that window, and the resulting gaps in the 10 ms grid are invisible: the logger keeps producing well-formed CSV throughout. Bus separation removes the contention, pinning the display to core 1 removes the CPU contention, and the firmware additionally never touches the display while a capture is active. Three independent defences against one silent failure.

Star grounding. Every ground returns to one point, and the LED ring's return gets its own wire rather than being daisy-chained. Ring current through a shared ground wire creates offsets that couple into the sensor as noise — and with a fusion sensor that noise does not merely add to the output, it degrades the orientation estimate that the entire feature representation is built on.

Why GPIO27 for the trigger. It is not a strapping pin. GPIO 0, 2, 12 and 15 are sampled at reset to select boot mode, so a trigger on any of those would drop the board into firmware download mode if a judge happened to be holding it at power-on.

3.5 Physical stack-up along the shaft R4

The wand is a 500 mm tube. Where each component sits along it is a mechanical decision with a signal consequence, so the layout is recorded here as a measurement, and the same numbers appear in `shared/wand_config.h` where the software can read them.



Why the BNO055 goes at 37 cm and not next to the ESP32. Three reasons, in order of weight. Signal: acceleration at the sensor scales with the lever arm, so a sensor near the hand sees a fraction of the motion and the accelerometer channels become mostly noise. Interference: the ESP32's WiFi front end, the boost converter's switching node and the battery's

charge current are all at the butt, and the BNO055's fusion output degrades with supply noise rather than merely getting noisier. Rigidity: the sensor must not move relative to the shaft, and a short unsupported span near the tip is easier to bond solidly than a crowded compartment.

Why the LED ring goes past it, at the tip. The ring is the feedback the caster and the judge actually look at, and at the tip it reads as the wand casting rather than as a board blinking. The cost is that its supply and data wires run the length of the tube, past the sensor. They are twisted with their own return and the ring's ground goes back to the star point on its own wire, because ~150 mA of switched LED current sharing a ground with the sensor is exactly the disturbance that shows up as orientation drift.

Why the OLED sits at 22 cm. It has to face the caster while the wand is held, which puts it in the middle third; further forward and the hand covers it, further back and it points at the floor when the wand is raised. It is also the heaviest thing that is not the battery, so keeping it behind the sensor keeps the balance point behind the grip.

TWO MOUNTING FACTS THE SOFTWARE CANNOT CHECK FOR ITSELF

The sensor's **+X must point at the tip** and the board must be **rigidly bonded**. If +X points at the butt, every derivation in §4.2 describes a mirrored wand; if the board can shift a few degrees in its mount, the anchor drifts between the pre-roll and the cast. Neither shows up in the data as anything but slightly worse accuracy. Test 5 of the bring-up suite (§13) checks the axis directly, by pointing the tip at the floor and reading the gravity vector out of the quaternion — body-frame "up" should come back as approximately $(-1, 0, 0)$.

4 Spell vocabulary, kinematic definitions, and the lever-arm model

Directional definitions are fixed in the wand's own body frame: +**X** along the wand towards the tip, +**Y** to the caster's left, +**Z** up through the back of the wand. The sensor is mounted with X along the shaft and Z out of the PCB face, 370 mm forward of the butt (§3.5). One sign convention is worth stating explicitly because it catches everyone: by the right-hand rule about +Y with +Z up, a *positive* pitch rate carries the tip *downward*. Every "tip up" move in this document therefore carries a negative pitch excursion, and the code is written that way rather than with a flipped axis, so that the frame stays the one the firmware uses.

#	Spell	Trajectory	Dominant signature	Nominal duration
0	Lumos	Upward loop	Sustained $+\omega_y$ sweeping a full 2π in the vertical plane, preceded by a lift	0.75–1.35 s
1	Alohomora	Backward loop and flick	$-\omega_y$ loop (opposite sign to Lumos) terminated by a short high-rate ω_z burst	0.80–1.40 s
2	Expelliarmus	Spiral swirl	Simultaneous ω_x and ω_y in phase quadrature; the longest and most sustained	1.00–1.60 s
3	Wingardium Leviosa	Swish and flick	Broad lateral ω_z sweep followed by a crisp upward ω_y flick	0.70–1.25 s
4	Stupefy (bonus)	Ballistic forward thrust	Near-zero rotation, large $+a_x$ then a hard decelerating rebound	0.40–0.75 s
5	NOISE	Everything else	Six recorded sub-behaviours: pacing, scratching, gesticulating, setting down, dropping, aborted casts	0.6–1.9 s

4.1 Why Stupefy is the right bonus spell, and the trap it sets twice

Stupefy was not chosen for flavour. The four compulsory spells all consist of rotation, and three of the four contain a terminal flick, so they occupy a similar region of feature space and compete with one another. Stupefy is defined as a thrust with almost no rotation, which places it in an otherwise empty region: it adds a class without crowding the existing ones. It also provides a free integrity check — its peak angular rate should be several times smaller than every other spell's, and if that ordering ever inverts, the labels have been shuffled.

The same property that makes it a good bonus spell has now caused *two* separate false-rejection defects in the plausibility gate, both caught on the bench and both documented in §8.3. Anything that pre-filters on "is this energetic enough to be a spell" will, if written carelessly, silently delete the one spell whose signature is not rotational.

4.2 The wand is a lever, and that dominates the signal R4

Everything above describes what the caster does. What the *sensor* records is a different quantity, and the difference is the 370 mm of tube between the two. For a point rigidly attached to a rotating body, its acceleration is

$$a_{\text{sensor}} = a_{\text{pivot}} + \alpha \times r + \omega \times (\omega \times r)$$

a_{pivot}
the hand travels

$\alpha \times r$
Euler
(angular accel)

$\omega \times (\omega \times r)$
centripetal
(always inward, always present)

where $\mathbf{r}$ is the fixed vector from the centre of rotation to the sensor, and all three terms are expressed in the sensor's own frame. The BNO055's linear-acceleration registers report exactly this quantity, because the fusion has already removed gravity.

The centre of rotation is a property of the *caster*, not the wand. A flick pivots at the wrist, roughly 80 mm behind the butt; a loop or a swish pivots at the elbow, roughly 330 mm behind it; walking swings the whole arm from the shoulder. Adding the sensor's 370 mm gives lever arms of 0.45 m, 0.70 m and 0.99 m — so even the *shortest* puts the BNO055 nearly half a metre from the point it is rotating about.

Put numbers to it. At the 6 rad/s a brisk flick reaches, the centripetal term alone is $0.45 \times 6^2 = 16 \text{ m/s}^2$, or **1.7 g** — from rotation, with the hand held perfectly still. The Euler term is larger still during the flick itself, because a 0.8 rad turn in 280 ms implies about 60 rad/s² and 0.45 m of arm turns that into 2.7 g. Meanwhile the caster's hand, travelling 200 mm in half a second, contributes about 0.2 g.

Term	Typical contribution	Reading
Centripetal $\omega \times (\omega \times \mathbf{r})$	1.0–2.5 g	Present whenever the wand rotates. Cannot be switched off, scales as ω^2 .
Euler $\alpha \times \mathbf{r}$	1.5–4.0 g	Peaks at the start and end of every sub-move. This is what a "flick" is, in the accelerometer.
Hand travel a_{pivot}	0.1–0.4 g	Dominant for Stupefy only, which is a thrust rather than a turn.

Three things follow that are not obvious without doing the arithmetic:

- **The accelerometer is mostly a second view of the gyroscope.** Within each rotational class the correlation between peak $|\omega|$ and peak $|a|$ measures 0.63–0.93 (§9.5). The channels are not independent, and a generator that made them independent was teaching the network a relationship the hardware will never show it.
- **Saturation is normal, not a failure.** A wand this long genuinely drives the $\pm 4 \text{ g}$ rail on sharp casts. That is expected, it affects Alohomora and Wingardium Leviosa most, and it is precisely why the impact test keys on duration instead of magnitude (§8.3).
- **Grip position matters more than grip angle.** A caster who chokes up on the shaft, or who flicks from the wrist where another swings from the elbow, changes $\mathbf{r}$ and therefore changes the acceleration amplitude by up to a factor of two — while the gyro channels barely move. That is modelled as a per-caster pivot scale in the bootstrap corpus and is one of the axes leave-one-caster-out validation is testing.

WHERE THIS MODEL IS APPROXIMATE, STATED PLAINLY

The pivot is treated as a single point on the wand's own axis, fixed for the duration of a gesture. Neither is exactly true: a real wrist sits slightly off the shaft's line, and a gesture that begins at the elbow and ends at the wrist moves its pivot mid-cast (Alohomora does exactly this, and is modelled with a pivot halfway between the two). Both approximations understate variety rather than inventing it, which is the safe direction. The quantity they get right — that the lever term dominates, and by how much — is the one the design depends on.

5 Data acquisition

5.1 Deterministic 100 Hz sampling

A hardware timer fires every 10,000 μs and does exactly one thing: release a binary semaphore. A FreeRTOS task pinned to core 0 at maximum priority takes that semaphore and performs the I2C burst.

`firmware/wand_infer/wand_infer.cpp`

```

// Note this task does the I2C read itself rather than doing it in the ISR. The
// 26-byte burst at 100 kHz is a blocking bus transaction of ~2.4 ms; performing
// it inside an ISR would block every other interrupt on the core for that whole
// time. The ISR does the one thing an ISR should do -- release a semaphore --
// and the task does the work.
void imuTask(void*) {
    uint32_t last = micros();
    RawSample prev{};
    for (;;) {
        xSemaphoreTake(s_tick, portMAX_DELAY);
        RawSample s;
        if (!bno::read(&s)) continue;
        const uint32_t now = micros();
        s.dt_us = now - last;
        last = now;

        // The BNO055's fusion output is also 100 Hz but its clock is not phase-locked
        // to ours, so the two beat and a frame is occasionally read twice. Harmless
        // -- at most one sample period of sample-and-hold, far below the noise floor
        // once the gesture is resampled to 64 steps -- but counted, because a HIGH
        // duplicate rate means something else is wrong: a bus hang, or the chip
        // having fallen back to CONFIG mode.
        s.dup = (s.qw == prev.qw && s.qx == prev.qx && s.qy == prev.qy &&
                 s.qz == prev.qz && s.lax == prev.lax && s.lay == prev.lay &&
                 s.laz == prev.laz && s.gx == prev.gx && s.gy == prev.gy &&
                 s.gz == prev.gz) ? 1 : 0;
        if (s.dup) ++s_dups;
        prev = s;

        s_ring[s_head] = s;
        s_head = (s_head + 1) % RING_LEN;
        ++s_seq;
    }
}

```

The division of labour matters more here than it did with the MPU-6050. The 26-byte burst at 100 kHz is a blocking transaction of roughly 2.4 ms — nearly a quarter of the sample period. Performing that inside the ISR would block every other interrupt on the core for the whole time. The ISR does the one thing an interrupt handler should do, and the task does the work.

5.2 One burst, one fusion frame R3

`firmware/common/bno055_fusion.cpp`

```

// One transaction, GYR_DATA_X_LSB (0x14) through LIA_DATA_Z_MSB (0x2D).
// 26 bytes covers gyro, Euler, quaternion and linear acceleration in a single
// burst, which guarantees every channel in a sample comes from the same fusion
// frame. The six Euler bytes are read and discarded -- they sit in the middle of
// the range so they cannot be skipped, and paying 0.5 ms for them is cheaper
// than splitting this into two transactions that could straddle a frame boundary.
bool read(RawSample* s) {
    const uint32_t t0 = micros();
    Wire.beginTransaction(I2C_ADDR_BNO);
    Wire.write(BNO_REG_GYR_DATA);
    if (Wire.endTransmission(false) != 0) { ++s_errors; return false; }
    if (Wire.requestFrom((int)I2C_ADDR_BNO, 26) != 26) { ++s_errors; return false; }

    auto rd = []() -> int16_t {
        const uint8_t lo = Wire.read(); // BNO055 is little-endian: LSB first
        return (int16_t)((uint16_t)Wire.read() << 8 | lo);
    };
    s->gx = rd(); s->gy = rd(); s->gz = rd(); // 0x14..0x19
    (void)rd(); (void)rd(); (void)rd(); // 0x1A..0x1F Euler
    s->qw = rd(); s->qx = rd(); s->qy = rd(); s->qz = rd(); // 0x20..0x27
    s->lax = rd(); s->lay = rd(); s->laz = rd(); // 0x28..0x2D
    s_last_us = micros() - t0;
    return true;
}

```

Reading gyro, quaternion and linear acceleration in *one* transaction is not merely faster than three. It guarantees that every channel in a sample comes from the same fusion frame. With three separate reads the quaternion and the acceleration can straddle a frame boundary, producing a skew that varies with bus timing and cannot be corrected afterwards — and in a representation where acceleration is *rotated* by the quaternion, a one-frame skew is not a small error, it is a rotation applied to the wrong vector.

The six Euler bytes sit in the middle of the address range and so cannot be skipped in a burst. They are read and discarded; paying 0.5 ms for them is cheaper than splitting the transaction.

5.3 Three data-quality gates the sensor swap made necessary

Gyro calibration status. The fusion's continuously-estimated gyro bias is what replaced the software bias subtraction. Below `CALIB_STAT` gyro level 3 that estimate is not trustworthy, and every capture taken there is quietly wrong with nothing to indicate it. The firmware therefore refuses to arm until the level reaches 3 and shows the progress on the OLED, which turns an invisible precondition into something the caster can see and wait for.

firmware/common/bno055_fusion.cpp

```

// CALIB_STAT (0x35): bits 7:6 SYS, 5:4 GYR, 3:2 ACC, 1:0 MAG.
// Only the gyroscopes field matters here. The fusion's continuously-estimated
// gyro bias is what replaced the pre-roll bias subtraction the MPU6050 build
// needed, and below status 3 that estimate is not yet trustworthy -- so every
// capture taken there is quietly wrong, with nothing to indicate it.
uint8_t gyroCalib() {
    s_calib = r8(BNO_REG_CALIB_STAT);
    return (uint8_t)((s_calib >> 4) & 0x03);
}

```

Duplicate fusion frames. The BNO055's fusion output is also 100 Hz, but its clock is not phase-locked to ours, so the two beat against each other and a frame is occasionally read twice. This is harmless — at most one sample period of sample-and-hold, far below the noise floor once the gesture is resampled to 64 steps — but it is counted, because a *high* duplicate rate means something else is wrong: a stalling bus, or the chip having fallen out of fusion mode.

A quaternion that never moves. If the chip drops back to CONFIG mode, or is left in a non-fusion mode, the quaternion registers read constant or zero. The tensor would still have the right shape, the model would still train, and four of its

twelve input channels would simply be dead. Nobody would notice. The collection script rejects any capture whose quaternion does not change.

training/collect.py

```
def validate(rows: list[str]) -> tuple[bool, str]:
    if len(rows) < C.MIN_GESTURE_SAMPLES:
        return False, f"only {len(rows)} samples (< {C.MIN_GESTURE_SAMPLES})"
    if len(rows) > C.MAX_GESTURE_SAMPLES + C.PREROLL_SAMPLES + C.POSTROLL_SAMPLES:
        return False, f"{len(rows)} samples, longer than the 2 s window"

    bad = dups = 0
    for r in rows[1:]:
        parts = r.split(",")
        # idx, qw, qx, qy, qz, lax, lay, laz, gx, gy, gz, dt_us, dup
        if len(parts) < 13:
            return False, f"malformed row: {len(parts)} columns, expected >= 13"
        if abs(int(parts[11]) - C.DT_US) > C.DT_TOL_US:
            bad += 1
        if int(parts[12]):
            dups += 1

    if bad:
        return False, (f"{bad}/{len(rows)} samples outside {C.DT_US}+/-{C.DT_TOL_US} us "
                       f"-- something is refreshing the LED or OLED during capture")

    frac = dups / max(len(rows) - 1, 1)
    if frac > C.MAX_DUP_FRACTION:
        return False, (f"{frac*100:.0f}% duplicate fusion frames (limit "
                       f"{C.MAX_DUP_FRACTION*100:.0f}%) -- the I2C bus is stalling or "
                       f"the BNO055 has dropped out of fusion mode")

    # A quaternion that never moves means the chip is not fusing: either it fell
    # back to CONFIG mode, or it is in a non-fusion mode where the QUA registers
    # read zero. Either way the orientation channels would be constant, and a
    # model trained on that has four dead inputs and nobody would notice.
    q = np.array([[int(v) for v in r.split(",")[1:5]] for r in rows], dtype=float)
    if np.abs(q - q[0]).max() < 1.0:
        return False, "quaternion never changes -- the BNO055 is not in fusion mode"
    if np.linalg.norm(q, axis=1).max() < 0.5 * C.BNO_LSB_PER_QUAT:
        return False, "quaternion norm is near zero -- no valid fusion output"

    return True, f"ok ({dups} duplicate frames)"
```

5.4 Segmentation, and why the pre-roll matters twice as much now

The trigger button defines the gesture window: press and hold, cast, release. Behind it a 320-sample circular buffer runs continuously so the window can be extended *backwards* in time.

In R2 the pre-roll existed for one reason: casters begin moving fractionally before their thumb lands, and without it the first fraction of every gesture is clipped — consistently, in training and inference alike, so nothing ever looks broken. The model simply learns a truncated gesture.

In R3 it carries a second and heavier load. **Sample 0 of the window is the anchor for the entire orientation representation.** Every quaternion channel, and the frame that acceleration and angular rate are expressed in, is defined relative to it. It therefore needs to be taken while the caster is still holding their resting grip, not mid-motion, and the 200 ms pre-roll is what guarantees that.

5.5 Collection protocol: manufacturing variation deliberately

Left to themselves, people converge on one comfortable way of performing a gesture within about five repetitions. A dataset with one style per caster is precisely the overfitting the invariance test is designed to expose, so the collection script

randomises the prompt for every repetition.

training/collect.py

```
if label == int(C.WandGestureClass.NOISE):
    prompt = rng.choice([
        "walk three steps holding the wand normally",
        "scratch your head with the wand in hand",
        "talk with your hands for two seconds",
        "put the wand down on the table",
        "start a spell and abandon it halfway",
        "pass the wand to your other hand",
    ])
    print(f"\n[{saved+1}/{args.reps}] NOISE -- {prompt}")
else:
    print(f"\n[{saved+1}/{args.reps}] {args.gesture} "
          f"speed={rng.choice(SPEEDS)} grip={rng.choice(GRIPS)} "
          f"pose={rng.choice(POSES)}")
    print("    hold the trigger, perform, release ...", end="", flush=True)
```

Axis of variation	Levels	What it defends against
Speed	slow / normal / fast	Judges cast at their own tempo, not yours
Grip	standard / rolled wrist / fingertip	The dominant between-person difference; anchoring does <i>not</i> remove it (§7.4)
Pose	standing / seated / arm extended	Changes the resting orientation the anchor is taken from
Caster	5 team members + 1 held out entirely	Leave-one-caster-out validation (§9.1)

Dataset budget

Target: 5 casters × 5 spells × 15 repetitions = 375 spell captures, plus 5 × 30 = 150 NOISE captures, for 525 total. At roughly 8 seconds per capture including the prompt, that is about 70 minutes of pure recording. A sixth person's data is collected but never used for training, and serves as a final untouched hold-out.

The NOISE script is drawn directly from the problem statement's own list of false-positive behaviours, plus two additions that matter more than any of them: half-completed spells, and casts begun and abandoned. Those are what a nervous judge actually produces, and no unsupervised outlier criterion will ever catch them, because a half-completed Lumos is a perfectly normal-looking signal that simply stops early.

NOISE HAS TO BE COLLECTED THE SAME WAY SPELLS ARE

Every NOISE capture is recorded with the trigger deliberately held down through the behaviour. Leaving the logger running and calling whatever it records "noise" produces nothing usable, because the trigger is what defines a window — and the false positives that actually matter are the ones where a judge *did* press the button and then did something that was not a spell.

5.6 The collection firmware, and one wire format R4

firmware/wand_logger/ shares the sampler, the driver and every constant with the demo firmware. That is a requirement rather than a convenience: the model is trained on what the logger writes to the serial port and runs on what wand_infer.cpp reads out of its ring buffer, so any difference between the two is a distribution shift that no downstream test can see.

The CSV it emits is the contract. training/collect.py parses exactly this and nothing else, and parity_host reads the same rows back — change one column here and three other files are wrong with no compiler to say so.


```
// ===== emission =====
// The CSV contract. training/collect.py parses exactly this and nothing else;
// firmware/wand_infer/parity_host.cpp reads the same rows back. Change one
// column here and three other files are wrong with no compiler to say so.
static void emit(int start_seq, int count, uint8_t cal) {
    Serial.printf("# GESTURE_START label=%d caster=%s rep=%d nsamples=%d "
        "calib=%u mode=0x%02X\n",
        s_label, s_caster, s_rep[s_label], count, cal, BNO_MODE_IMU);
    Serial.println(F("idx,qw,qx,qy,qz,lax,lay,laz,gx,gy,gz,dt_us,dup"));
    int idx = ((start_seq % RING_LEN) + RING_LEN) % RING_LEN;
    for (int i = 0; i < count; ++i) {
        const RawSample& s = s_ring[idx];
        Serial.printf("%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%lu,%u\n",
            i, s.qw, s.qx, s.qy, s.qz, s.lax, s.lay, s.laz,
            s.gx, s.gy, s.gz, (unsigned long)s.dt_us, s.dup);
        idx = (idx + 1) % RING_LEN;
    }
    Serial.println(F("# GESTURE_END"));
}
```

The command channel runs the other way and is the same kind of contract: `collect.py` already sends `SET label=<n> caster=<name>`, so the firmware parses that rather than inventing a scheme of its own. A mismatch there does not fail — it files every capture under the wrong label, quietly, and the first symptom is a confusion matrix nobody can explain.

6 The signal processing chain

The chain converts a variable-length raw integer stream into a fixed 64×12 float tensor. It is implemented twice — once in Python as the reference, once in C++ for the device — and the two are asserted equal to 10^{-3} by an automated test (§9.2).

6.1 What the sensor already did, and why we let it R3

Three stages that R2 performed in software are gone. The BNO055's Cortex-M0 removes gyro bias continuously, runs a Kalman-class orientation estimator, and subtracts gravity — all three better than a 100 Hz complementary filter written by us, and all three for free in terms of ESP32 cycles.

The advice we were given was the opposite: put the chip in AMG (non-fusion) mode so the existing chain would not subtract gravity twice. The hazard is real — double subtraction produces no error message anywhere — but disabling the fusion is the wrong resolution. Three things are gained by deleting our chain instead:

- **Latency.** The complementary filter and gravity projection cost roughly 920 transcendental calls per gesture, about 2.5 ms on Xtensa. The R3 chain contains no transcendental function at all on the hot path except the two `sqrtf` calls per sample for the magnitudes.
- **Correctness.** Our filter recovered roll and pitch only; yaw is unobservable from accelerometer and gyroscope alone in that formulation. The chip's estimator produces a full quaternion.
- **Risk.** The complementary filter's initialisation is, on the evidence of our own negative control (§9.2), the single easiest thing in this project to get subtly wrong. Deleting it deletes that risk.

What we give up is accelerometer range: AMG mode would permit ± 16 g, fusion locks ± 4 g. That trade is examined honestly in §8.3, and the conclusion is that clipping a thrust does not make it less distinguishable — it makes it *more* so, because no other class saturates.

6.2 Step 1: raw counts to physical units

training/preprocess.py

```
# Step 1 -- raw counts to physical units
# -----
def to_physical(raw: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """raw: (N, 10) int16 columns qw qx qy qz lax lay laz gx gy gz.

    Returns (quaternion, linear acceleration in g, angular rate in rad/s).

    The divisors are fixed by the BNO055 datasheet and by the UNIT_SEL register,
    which the firmware writes explicitly at boot. This is a real improvement over
    the MPU6050, where the scale depended on a range register that could be
    changed without anyone updating the constant -- here there is no range
    register to get wrong, because the fusion algorithm owns it.
    """
    raw = np.asarray(raw, dtype=np.float64)
    quat = raw[:, 0:4] / C.BNO_LSB_PER_QUAT
    acc = raw[:, 4:7] / C.LINACC_LSB_PER_G # m/s^2 counts -> g
    gyro = (raw[:, 7:10] / C.BNO_LSB_PER_DPS) * C.DEG2RAD # dps counts -> rad/s
    return quat, acc, gyro
```

6.3 Step 2: anchoring — the idea the whole representation rests on R3

The chip reports linear acceleration and angular rate in the *sensor body frame* — a frame that rotates with the wand as the gesture is performed. That is the natural output and it is also the problem: a constant push in the world appears as a time-

varying signal in a spinning frame, so translation and rotation arrive at the network tangled together, and the network has to spend capacity disentangling them before it can start recognising anything.

Because we also have the orientation, we can undo that. Define

```
q_rel[i] = conj(q[0]) * q[i]

a_anchored[i] = rotate(q_rel[i], a_body[i])
w_anchored[i] = rotate(q_rel[i], w_body[i])
```

and every quantity is now expressed in the single fixed frame the wand occupied at sample 0. Channels 0–2 become pure translation in a frame that does not spin; channels 3–9 carry the rotation, explicitly and separately. The two are decoupled.

training/preprocess.py

```
# Step 2 -- anchor the gesture to its own starting orientation
# -----
def anchor(quat: np.ndarray) -> np.ndarray:
    """q_rel[i] = conj(q[0]) * q[i] -- orientation relative to gesture start.

    q[0] is the first PRE-ROLL sample, 200 ms before the trigger went down, when
    the caster is still holding position. That is the best available description
    of "how this person was holding the wand before they started", which is
    exactly the reference we want to remove.

    q_rel[0] is the identity by construction, so channel 6 always starts at 1.0
    and channels 7..9 always start at 0.0. That fixed anchor is genuinely useful
    to the network: every gesture begins at the same point in the representation,
    so the convolutions only ever have to model the DEPARTURE from it.
    """
    return quat_mul(quat_conj(quat[0])[None, :], quat)
```

There is a useful side effect. `q_rel[0]` is the identity by construction, so channel 6 always starts at exactly 1.0 and channels 7–9 at exactly 0.0. Every gesture begins at the same point in the representation, and the convolutions only ever have to model the *departure* from it. That fixed starting point is relied on downstream: the augmentation pipeline re-establishes it after every transform (§7.4), and a unit test asserts the channel layout that makes it meaningful.

WHAT ANCHORING DOES AND DOES NOT BUY — STATED PRECISELY

Anchoring removes **which way the caster was facing**. It does **not** remove **how they were holding the wand**, because the anchor rotates with the grip: a caster whose wrist is rolled 30° further has an anchor rolled 30° further too. Grip variation is handled separately, by the augmentation in §7.4, and the two mechanisms are complementary rather than redundant. It would be easy to overclaim here and we are not going to — the invariance test will be run by people whose grips we have never seen.

6.4 Quaternion sanitising: three hazards that all look like valid data

training/preprocess.py

```
def sanitise_quaternions(q: np.ndarray) -> np.ndarray:
    """Normalise, replace degenerate quaternions with the identity, and enforce
    hemisphere continuity along the time axis.

    Three separate hazards, all of which produce a valid-looking tensor:

    1. A quaternion read while the BN0055 is still in CONFIG mode, or during the
    I2C hiccup that clock stretching can cause, comes back as all zeros.
    Normalising that is a division by zero. We substitute the identity, and
    because the identity means "no rotation relative to the reference" it is
    the safest possible fallback -- the sample simply contributes no rotation.

    2. Quaternions double-cover rotations:  $q$  and  $-q$  are the same orientation.
    The BN0055 is free to hand back either, and it does flip sign. Without
    the continuity pass, channels 6..9 contain sign discontinuities that look
    like instantaneous 360-degree rotations, and no convolution can learn
    through that.

    3. Continuity is enforced HERE, before the anchoring, because quaternion
    multiplication is bilinear -- a sign fixed now stays fixed in  $q_{rel}$ .
    """
    q = np.asarray(q, dtype=np.float64).copy()
    norm = np.linalg.norm(q, axis=1)

    bad = norm < C.QUAT_MIN_NORM
    q[bad] = np.array([1.0, 0.0, 0.0, 0.0])
    norm[bad] = 1.0
    q /= norm[:, None]

    # Hemisphere continuity: keep every sample on the same side as the one before.
    for i in range(1, len(q)):
        if float(np.dot(q[i], q[i - 1])) < 0.0:
            q[i] = -q[i]
    return q
```

The hemisphere continuity pass is the one that would be easiest to omit and hardest to debug. A quaternion double-covers rotations — q and $-q$ are the same orientation — and the chip is free to return either. Without the continuity pass, channels 6–9 contain sign discontinuities that look like instantaneous 360-degree rotations, and no convolution can learn through that. Our negative control for this exact omission produced a parity divergence of 1.92, roughly two thousand times the tolerance (§9.2).

6.5 Steps 3 and 4: resampling and normalisation

training/preprocess.py

```

# Step 3 -- resampling
# -----
def resample_fixed(x: np.ndarray, n_out: int = C.N_RESAMPLE) -> np.ndarray:
    """Linear resample (N, C) -> (n_out, C) on a normalised index axis.

    Index-space rather than time-space interpolation, because the logger already
    guarantees a uniform 10 ms grid -- it rejects any capture whose dt_us leaves
    the +/- 200 us tolerance. Index space is exactly reproducible in C++ with two
    multiplies and an add, with no floating-point time accumulation to diverge.

    This step is what gives speed invariance: a 0.6 s Lumos and a 1.4 s Lumos both
    become 64 steps, so the network sees the SHAPE of the trajectory and is blind
    to how fast the judge happened to wave. Duration is deliberately NOT fed to
    the network -- it is checked separately, as a hard range, in the Stage 1 gate.

    The quaternion channels are interpolated linearly rather than by slerp, and
    are deliberately NOT renormalised afterwards. Two reasons: between adjacent
    100 Hz samples the rotation is under a degree, where lerp and slerp differ by
    less than 1e-5; and a renormalisation step is one more place for the Python
    and C++ chains to disagree, for a correction smaller than the sensor noise.
    """
    n_in = len(x)
    if n_in == n_out:
        return x.astype(np.float64).copy()
    out = np.zeros((n_out, x.shape[1]), dtype=np.float64)
    step = (n_in - 1) / (n_out - 1)
    for k in range(n_out):
        pos = k * step
        i0 = int(np.floor(pos))
        i1 = min(i0 + 1, n_in - 1)
        f = pos - i0
        out[k] = x[i0] * (1.0 - f) + x[i1] * f
    return out

```

Resampling produces speed invariance: a 0.6-second Lumos and a 1.4-second Lumos both become 64 steps, so the network sees the *shape* of the trajectory and is structurally blind to how fast the judge happened to wave. Interpolation is in index space rather than time space, because the logger already guarantees a uniform 10 ms grid and index space reduces to two multiplies and an add in C++, with no floating-point time accumulation that could diverge between implementations.

The quaternion channels are interpolated linearly rather than by slerp, and deliberately not renormalised afterwards. Between adjacent 100 Hz samples the rotation is under a degree, where lerp and slerp differ by less than 10^{-5} ; a renormalisation step would be one more place for the two chains to disagree, in exchange for a correction smaller than the sensor noise.

training/preprocess.py

```
def normalise(x: np.ndarray) -> np.ndarray:
    """Divide each channel group by its frozen constant and clip to [-1, 1].

    Deliberately NOT a per-sample z-score. Two reasons:

    1. A per-sample z-score would normalise away amplitude, and amplitude is a
       real discriminative feature here -- Stupefy is a hard ballistic thrust and
       Wingardium Leviosa is a gentle swish. Z-scoring makes them look alike.
    2. A per-sample statistic means the MCU must buffer the whole gesture, compute
       mean and variance, then walk it again. Fixed divisors are one multiply per
       sample and can run streaming.

    The quaternion channels need no divisor at all -- a unit quaternion already
    lives in [-1, 1] on every component. That is a small but real bonus of
    carrying orientation as a quaternion rather than as Euler angles or a rotation
    vector: it is natively in the range int8 quantisation wants.
    """
    out = np.empty_like(x)
    out[:, C.CH_ACC:C.CH_ACC + 3] = np.clip(x[:, C.CH_ACC:C.CH_ACC + 3] / C.NORM_ACC_LIN, -1.0, 1.0)
    out[:, C.CH_GYRO:C.CH_GYRO + 3] = np.clip(x[:, C.CH_GYRO:C.CH_GYRO + 3] / C.NORM_GYRO, -1.0, 1.0)
    out[:, C.CH_QUAT:C.CH_QUAT + 4] = np.clip(x[:, C.CH_QUAT:C.CH_QUAT + 4], -1.0, 1.0)
    out[:, C.CH_AMAG] = np.clip(x[:, C.CH_AMAG] / C.NORM_AMAG, 0.0, 1.0)
    out[:, C.CH_WMAG] = np.clip(x[:, C.CH_WMAG] / C.NORM_WMAG, 0.0, 1.0)
    return out
```

Normalisation is deliberately not a per-sample z-score. Amplitude is a real discriminative feature here — Stupefy is a hard ballistic thrust and Wingardium Leviosa is a gentle swish, and z-scoring makes them look alike — and a per-sample statistic would force the MCU to buffer the gesture, compute mean and variance, and walk it again. Fixed divisors are one multiply per sample.

The divisor that was derived with the sign the wrong way round R4

R3 set `NORM_ACC_LIN = 3.0` and wrote a unit test asserting it, on this reasoning: the fusion locks the accelerometer to ± 4 g, gravity consumes one of them, therefore 3 g is the ceiling. The reasoning is confident, it has an assertion behind it, and it is wrong. Gravity is *subtracted* by the fusion, not spent from the budget. The rail applies to the raw signal, before that subtraction:

```
lin_axis = raw_axis - g_axis,    raw_axis ∈ [-4, +4] g,    g_axis ∈ [-1, +1] g
    ⇒ lin_axis ∈ [-5, +5] g
```

So the reportable range is the rail *plus* one gravity. The measured per-axis maximum across the bootstrap corpus is **exactly 5.00 g** — the arithmetic above, reached and not exceeded, which is as clean a confirmation as this kind of derivation gets.

This mattered. At 3.0 the tensor railed on **2.4% of all samples**, and not at random: the samples that clip are the peaks of the sharp flicks, which is the part of the trace that identifies Alohomora and Wingardium Leviosa. The divisor is now 5.0 — a hard physical bound, so nothing clips and no int8 code is spent on a value the sensor cannot produce — and railing fell to 0.08%. The unit test was rewritten to assert the corrected relationship, and it is worth noting that the old test *passed* throughout: an assertion is only as good as the derivation behind it, and this one had been guarding the wrong inequality with complete conviction.

The gyro divisor was re-examined in the same pass and deliberately left alone. The corpus reaches only 0.43 of `NORM_GYRO` at the 99th percentile, so tightening it to 9 or 10 would buy ~20% more int8 resolution on the quiet part of every gesture. It would buy that by railing the samples out at the tail — which are the peaks of the flicks, and the presence and sharpness of a terminal flick is the entire difference between Alohomora and Lumos. Measured, considered, unchanged.

6.6 The resulting tensor

Ch	Name	Frame	Divisor	Role
0–2	a_x, a_y, a_z linear	anchored	5.0 g	Pure translation, gravity removed by the sensor
3–5	$\omega_x, \omega_y, \omega_z$	anchored	12.0 rad/s	Rotation rate, bias removed by the fusion
6–9	q_w, q_x, q_y, q_z relative	—	<i>none</i>	The orientation trajectory itself. A loop is a path through orientation space.
10	$ a_{lin} $	invariant	5.0 g	Rotation-invariant translational envelope
11	$ \omega $	invariant	12.0 rad/s	Rotation-invariant rotational envelope

The quaternion channels need no divisor at all: a unit quaternion already lives in $[-1, 1]$ on every component. That is a small but real bonus of carrying orientation as a quaternion rather than as Euler angles (which wrap) or a rotation vector (which folds at 180° , exactly where a Lumos loop passes).

Channels 10 and 11 are not redundant with the others. They are invariant under a change of grip, whereas channels 0–9 all transform under it. During augmentation the vector channels are rotated and the quaternion is conjugated while these two are left untouched, which gives the network a stable backbone to attach the transforming channels to.

7 The TinyML model

7.1 Architecture

training/train_cnn.py

```
# model
# -----
def build_model(n_classes: int = C.NUM_CLASSES) -> keras.Model:
    """(64, 8) -> (n_classes,) softmax.

    Layer-by-layer reasoning:

    Conv1D(16, k=5, stride=2)
        A 5-sample kernel spans 50 ms, which is roughly the shortest meaningful
        wrist event. Stride 2 halves the sequence immediately: at 100 Hz with a
        44 Hz anti-alias filter there is very little real information above 25 Hz,
        so the second half of the spectrum is noise we are paying to convolve.
        This one stride is the largest single latency saving in the network.

    Two SeparableConv1D(32, k=3) blocks, each followed by MaxPool(2)
        Separable = depthwise then pointwise. A dense Conv1D(32, k=3) over 32
        channels costs  $3 \times 32 \times 32 = 3072$  multiplies per step; separable costs
         $3 \times 32 + 32 \times 32 = 1120$ , a 2.7x saving for a negligible accuracy difference at
        this scale. Pooling drives  $32 \rightarrow 16 \rightarrow 8$  steps, so the final receptive
        field covers most of the gesture while the tensor stays tiny.

    Flatten, NOT GlobalAveragePooling
        This is a deliberate and load-bearing choice. GAP would discard *when*
        each feature occurred, and when is exactly what separates Alohomora
        (backward loop THEN flick) from Wingardium Leviosa (swish THEN flick) --
        both contain a loop-ish sweep and a flick, differing in order, direction
        and relative timing. A GAP model confuses them; a flatten model can see
        the ordering. The cost is  $256 \times 16 = 4096$  extra weights, which on a chip
        with 520 KB of SRAM is not a real cost.

    BatchNorm before the activation
        Folded into the preceding convolution's weights at conversion time, so it
        costs nothing at inference but materially stabilises training on a small,
        heavily augmented dataset.

    Dropout 0.3 / 0.2
        The dataset will be ~600 real captures. Without aggressive dropout the
        network memorises casters, and LOCO accuracy collapses while random-split
        accuracy stays flattering.
    """
    inp = keras.Input(shape=(C.N_RESAMPLE, C.N_CHANNELS), name="gesture")

    x = keras.layers.Conv1D(16, 5, strides=2, padding="same", use_bias=False)(inp)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.ReLU()(x)

    x = keras.layers.SeparableConv1D(32, 3, padding="same", use_bias=False)(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.ReLU()(x)
    x = keras.layers.MaxPooling1D(2)(x)

    x = keras.layers.SeparableConv1D(32, 3, padding="same", use_bias=False)(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.ReLU()(x)
    x = keras.layers.MaxPooling1D(2)(x)

    x = keras.layers.Flatten()(x)
    x = keras.layers.Dropout(0.3)(x)
    x = keras.layers.Dense(16, activation="relu")(x)
    x = keras.layers.Dropout(0.2)(x)
```



```

out = keras.layers.Dense(n_classes, activation="softmax", name="spell")(x)

return keras.Model(inp, out, name="wand_cnn")

```

Layer	Output	Params	Reasoning
Input	64×12	—	Anchored tensor from §6
Conv1D 16, k=5, s=2	32×16	960	A 5-sample kernel spans 50 ms, roughly the shortest meaningful wrist event. The stride halves the sequence immediately: the fusion's own bandwidth limit leaves little real content above 25 Hz, so the upper half of the spectrum is noise we would be paying to convolve. This single stride is the largest latency saving in the network.
SeparableConv1D 32, k=3	32×32	560	Depthwise then pointwise: $3 \times 16 + 16 \times 32 = 560$ multiplies per step against 1,536 for a dense Conv1D — a $2.7\times$ saving at negligible accuracy cost at this scale.
MaxPool 2	16×32	—	Halves the sequence; widens the effective receptive field
SeparableConv1D 32, k=3	16×32	1,120	Second stage; receptive field now covers most of the gesture
MaxPool 2	8×32	—	—
Flatten	256	—	See §7.2 — a load-bearing choice
Dropout 0.3 → Dense 16 → Dropout 0.2	16	4,112	Compression before the classifier. The dataset will be ~600 captures; without aggressive dropout the network memorises casters.
Dense 6, softmax	6	102	Five spells plus NOISE
Total (incl. BatchNorm statistics)		7,174	22,520 bytes as an int8 flatbuffer

Growing the input from 8 to 12 channels cost 320 parameters and 5,120 MAC operations — about 8% of the network. That is the entire price of carrying the orientation trajectory, and it is a bargain.

7.2 Flatten rather than global average pooling

Most TinyML classifiers of this size end in global average pooling, which would have saved 4,096 weights. It was rejected deliberately. Pooling discards *when* each feature occurred, and when is exactly what separates Alohomora (backward loop, *then* flick) from Wingardium Leviosa (lateral swish, *then* flick): both contain a broad sweep and a terminal flick, differing in order, direction and relative timing. A pooled representation sees "one sweep, one flick" for both; a flattened one retains the ordering across the eight remaining time steps.

The cost is 4,096 additional int8 weights, which on a chip with 520 KB of SRAM is not a real cost. This is a concrete instance of a general principle in this design: spend memory, which is abundant, to protect accuracy and latency, which are not.

7.3 Training regime

```
training/train_cnn.py
```

```
def compile_model(m: keras.Model, lr: float = 2e-3) -> keras.Model:
    m.compile(
        optimizer=keras.optimizers.Adam(lr),
        # Label smoothing 0.05: stops the network driving softmax to 1.0 on the
        # training casters. That over-confidence is what makes a rejection
        # threshold useless -- an over-confident model is confidently wrong on a
        # judge's hand, and every probability lands above any threshold you set.
        loss=keras.losses.CategoricalCrossentropy(label_smoothing=0.05),
        metrics=["accuracy"],
    )
    return m
```

Label smoothing at $\epsilon = 0.05$ prevents the network driving its softmax to 1.0 on the training casters. Over-confidence is not cosmetic: an over-confident model is confidently *wrong* on an unseen hand, and it makes every rejection threshold useless because all probabilities land above whatever value you pick. §8.4 documents the concrete consequence and how it is handled.

training/train_cnn.py

```
# class weights
# -----
def class_weights(y: np.ndarray) -> dict[int, float]:
    """NOISE is cheap to record and will be over-represented; inverse-frequency
    weighting stops the network taking the easy win of predicting NOISE often.
    The NOISE weight is then nudged UP by 1.25 anyway, because in this scoring
    scheme a missed spell costs a point and a misfire costs more."""
    counts = np.bincount(y, minlength=C.NUM_CLASSES).astype(np.float64)
    counts[counts == 0] = 1.0
    w = counts.sum() / (len(counts) * counts)
    w[int(C.WandGestureClass.NOISE)] *= 1.25
    return {i: float(v) for i, v in enumerate(w)}
```

Class weighting is inverse-frequency with the NOISE weight nudged up a further 1.25. That extra factor encodes the scoring asymmetry directly into the loss: under this problem statement a missed spell scores nothing while a misfire is penalised, so the training objective should not treat the two errors as equal.

7.4 Augmentation as manufactured invariance R3

With five casters there are five hand-writing styles in the dataset. The judges will be a sixth, seventh and eighth. That gap cannot be collected away in the time remaining, so augmentation manufactures the between-caster variation that could not be recorded. Every transform is physically exact rather than generic noise — an augmentation that produces impossible signals teaches the network to separate real from impossible instead of Lumos from Alohomora.

training/augment.py

```
def grip_rotation(x: np.ndarray, rng: np.random.Generator,
                 roll_deg: float = 25.0, off_deg: float = 10.0) -> np.ndarray:
    """Rotate the sensor's mounting frame -- i.e. simulate a different grip.
```

This is the highest-value augmentation for the invariance test. Two people holding the same wand differ mostly by rotation about its long axis (how far the wrist is rolled), and secondarily by small pitch and yaw offsets.

Under a constant grip change G , the anchored representation transforms as:

$$\begin{aligned} a' &= G a && \text{(and likewise for the angular rate)} \\ q_{\text{rel}}' &= G q_{\text{rel}} G^{-1} && \text{-- a CONJUGATION, not a left-multiply} \end{aligned}$$

The conjugation is the part worth being careful about. q_{rel} is a rotation expressed in the anchored frame, and rotating the frame itself is a change of basis, which for rotations is conjugation. Left-multiplying instead -- the obvious-looking mistake -- would compose an extra rotation into the gesture

and produce trajectories no wrist could perform.

Note also that anchoring does NOT make this augmentation redundant. Anchoring removes which way the caster was FACING; it does not remove how they were holding the wand, because the anchor rotates with the grip. The two are complementary, and the report says so explicitly rather than overclaiming.

```
"""
g = _quat_from_axis_angle(np.array([1.0, 0.0, 0.0]),
                           np.deg2rad(rng.uniform(-roll_deg, roll_deg)))
g = _qmul(g, _quat_from_axis_angle(np.array([0.0, 1.0, 0.0]),
                                   np.deg2rad(rng.uniform(-off_deg, off_deg))))
g = _qmul(g, _quat_from_axis_angle(np.array([0.0, 0.0, 1.0]),
                                   np.deg2rad(rng.uniform(-off_deg, off_deg))))
gc = _qconj(g)

out = x.copy()
n = x.shape[0]
gb = np.broadcast_to(g, (n, 4))
out[:, A0:A1] = _qrot(gb, x[:, A0:A1])
out[:, W0:W1] = _qrot(gb, x[:, W0:W1])
out[:, Q0:Q1] = _qmul(_qmul(gb, x[:, Q0:Q1]), np.broadcast_to(gc, (n, 4)))
return out
```

The conjugation is the part worth being careful about, and it is new in R3. `q_rel` is a rotation expressed in the anchored frame, and rotating the frame itself is a change of basis, which for rotations is conjugation — not a left-multiply. Left-multiplying, the obvious-looking mistake, would compose an extra rotation *into* the gesture and produce trajectories no wrist could perform, and the network would learn to detect the difference between real and augmented instead of the difference between spells.

Keeping the tensor internally consistent

Two further R3 subtleties, both of which would produce a self-contradictory tensor if ignored:

Scaling the rate without scaling the orientation. If the magnitude augmentation multiplies the angular-rate channels by 1.2 but leaves the quaternion channels alone, channels 3–5 describe a rotation that channels 6–9 do not perform. The network would happily learn to detect *that* contradiction. So the relative quaternion's rotation angle is scaled by the same factor, about the same axis.

Moving the anchor. A negative time shift drops early samples, so the new first sample is no longer the identity and the invariant that every real tensor satisfies is broken. Rather than forbidding the transform, a re-anchoring step runs once at the end of the pipeline and restores it.

```
training/augment.py
```

```
def reanchor(x: np.ndarray) -> np.ndarray:
    """Restore q_rel[0] = identity, rotating the vector channels to match.

    Applied once at the end of the pipeline, so that whatever the individual
    transforms did, the result satisfies the same invariant the real signal chain
    produces. Skipping this would let the network see augmented tensors whose
    first sample is not the identity and real tensors whose first sample always
    is -- a difference it could learn instead of the gesture.

    q'  = conj(q[0]) * q
    a'  = rotate(conj(q[0]), a)
    """
    out = x.copy()
    q = out[:, Q0:Q1]
    q0 = q[0] / (np.linalg.norm(q[0]) + 1e-12)
    q0c = _qconj(q0)
    n = x.shape[0]
    q0cb = np.broadcast_to(q0c, (n, 4))
    out[:, Q0:Q1] = _qmul(q0cb, q)
    out[:, A0:A1] = _grot(q0cb, out[:, A0:A1])
    out[:, W0:W1] = _grot(q0cb, out[:, W0:W1])
    return out
```

Transform	Range	Models
Grip rotation	$\pm 25^\circ$ about the wand axis, $\pm 10^\circ$ off-axis	Grip styles and wrist roll — the part anchoring does <i>not</i> remove
Non-uniform time warp	4 knots, $\sigma = 0.18$, endpoints pinned	Personal rhythm within a gesture. Uniform speed change is already removed by the resample, so only non-uniform warping is meaningful.
Magnitude scale	0.82–1.20, independent for accel and gyro, quaternion angle scaled to match	How forcefully and how widely the caster performs
Time shift	± 5 samples, edge-padded, then re-anchored	Trigger-timing variance
Sensor noise + residual bias	$\sigma_a=0.006$, $\sigma_\omega=0.004$, $\sigma_q=0.004$, bias $\sigma=0.006$	The fusion's own orientation estimation error, which grows while gyro calibration is imperfect. Training with it present is what makes the network tolerant of a sensor that has only just reached calibration status 3 — which, in a demo queue, it will have.

7.5 Int8 quantisation, and what it actually cost

training/train_cnn.py

```
# quantisation and export
# -----
def quantise(model: keras.Model, X_rep: np.ndarray) -> bytes:
    """Full-integer int8 quantisation, int8 in and int8 out.

    Why full-integer rather than float16 or dynamic-range:
    * TFLite-Micro on an ESP32 (Xtensa LX6, no FPU-accelerated SIMD for this)
      runs int8 kernels several times faster than float.
    * int8 in/out means the firmware never allocates a float input buffer --
      preprocessing writes quantised bytes straight into the arena.
    * Weights are 4x smaller, though on this chip that is a bonus, not the point.

    The representative dataset MUST come from the real training distribution. It
    is what fixes the activation scales; feeding it synthetic or augmented-only
    data sets scales that do not match what the sensor produces, and the model
    then saturates on the device while looking perfect in Python. This is the
    quiet failure mode that the parity check in P8 exists to catch.
    """
    def rep_gen():
        idx = np.random.default_rng(SEED).permutation(len(X_rep))[:400]
        for i in idx:
            yield [X_rep[i][None, ...].astype(np.float32)]

    conv = tf.lite.TFLiteConverter.from_keras_model(model)
    conv.optimizations = [tf.lite.Optimize.DEFAULT]
    conv.representative_dataset = rep_gen
    conv.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    conv.inference_input_type = tf.int8
    conv.inference_output_type = tf.int8
    return conv.convert()
```

Full-integer quantisation with int8 input *and* output was chosen over float16 or dynamic-range for three reasons: TFLM's int8 kernels run several times faster than float on the Xtensa LX6; int8 input means the firmware never allocates a float input buffer, because preprocessing writes quantised bytes straight into the arena; and weights are four times smaller, which here is a bonus rather than the point.

A twelve-channel tensor raises a legitimate concern that eight channels did not. TFLite uses a single *per-tensor* scale for the input, and the quaternion channels use the full $[-1, 1]$ range while the anchored acceleration channels are much smaller — so the acceleration channels could in principle be crushed to a handful of quantisation levels. We measured it rather than assuming either way:

Metric	Value	Reading
Input scale / zero point	0.007736 / -2	259 levels across $[-1, 1]$
Worst channel resolution	58 levels	<code>wz_anch</code> ; every other channel gets 60–232
float argmax accuracy	1.000	Quantisation costs nothing here. The concern was real and did not materialise.
int8 argmax accuracy	1.000	

The evaluation is deliberately run against the *quantised* model, because that is the artefact that ships. A float model at 92% and an int8 model at 86% is a six-point surprise that belongs in a lab, not on a stage.

THE REPRESENTATIVE DATASET IS NOT A FORMALITY

It is what fixes the activation scales. Feeding it synthetic or augmented-only data sets scales that do not match what the sensor actually produces; the model then saturates on the device while looking perfect in Python, with no error anywhere. When real captures land, the representative dataset must be regenerated from them — this is a step it is very easy to skip.

8 False-positive rejection: the Muggle Movement defence

8.1 Why a trained NOISE class, and not a confidence threshold

The obvious approach — train on four spells and reject anything below a softmax threshold — fails for a structural reason rather than a tuning reason. A softmax over four classes must sum to one. Shown a shrug, such a network has no way to express "none of these"; it is forced to distribute its belief across four spells, and it frequently lands on one of them with high confidence. No threshold fixes this, because the problem is that the model has nowhere to put the probability mass.

Giving NOISE its own trained output gives the network somewhere to put it. The difference is between a threshold hack applied to a model that cannot represent the concept, and a classifier that has actually learned what "not a spell" looks like — including what a half-completed spell looks like, which no unsupervised outlier criterion will ever catch.

8.2 The five-stage cascade

Stages are ordered cheapest first: there is no reason to spend milliseconds of inference establishing that someone scratched their nose. Every stage can only ever turn a fire into a no-fire; none can invent a cast.

Stage	Mechanism	Cost	Catches
0	Hold-to-record trigger	~0	Everything the caster did not deliberately mark as a cast
1	Kinematic plausibility	~5 μ s	Scratching, pacing, impacts, implausible durations
2	Trained NOISE class	~2–5 ms	Gesture-shaped motion that is not a spell; aborted casts
3	Calibrated confidence floor and top-1/top-2 margin	~2 μ s	Ambiguity between similar spells
4	Temporal persistence (auto-segment mode only)	2 windows	Single-frame flukes from a sliding window
5	500 ms refractory period	~1 μ s	Double-casting from a bounced trigger

The free-fall detector from R2 is gone, deliberately. In fusion mode a dropped wand's free-fall signature is not visible: linear acceleration is the raw reading minus the fusion's gravity estimate, and that estimate coasts during a fall, so a falling wand reports about 1 g of linear acceleration rather than zero. Reinstating it would have meant reading the gravity vector as well, adding six bytes and a register to every sample. It was removed instead, because two things already cover the case: a hold-to-record trigger means a dropped wand releases the button and ends its own capture, and the impact at the end of a drop is exactly the impulse the new spike test catches.

Stage 1 in full

firmware/wand_infer/reject.cpp

```
// Stage 1 -- kinematic plausibility.
//
// Each bound is tied to a specific behaviour from the PS's own false-positive
// list, which is deliberate: these are not generic sanity checks, they are
// answers to the named test cases.
// -----
RejectReason gateKinematics(const KinFeatures& k) {
    // No incantation is shorter than 300 ms of motion or longer than 2 s. This is
    // also where duration is enforced at all -- the network never sees it, because
    // the fixed 64-step resample makes the model deliberately speed-blind so that a
    // fast judge and a slow judge get the same answer. Bounds are on the whole
    // buffer, pre-roll and post-roll included.
    if (k.duration_s < REJ_MIN_DURATION_S) return REJ_TOO_SHORT;
    if (k.duration_s > REJ_MAX_DURATION_S) return REJ_TOO_LONG;
```

```

// Energy, as a DISJUNCTION. A window is plausible if it is rotationally
// energetic (the four wand-waving spells) OR ballistically energetic
// (Stupefy, which is a thrust and barely rotates at all).
//
// Getting this wrong is easy and expensive. The first version of this gate
// required rotation unconditionally and rejected every single Stupefy, on the
// bench, before the network ever saw one -- the bonus spell was unreachable
// and the fault looked like a model problem, not a gate problem.
//
// "scratching an itch" is fast but small: low peak rate AND low linear accel.
// "random pacing" moves the wand through space but barely rotates it, so the
// swept angle stays small AND there is no thrust. Neither passes either arm.
{
    const bool rotational = (k.peak_w >= REJ_MIN_PEAK_W) && (k.path_rad >= REJ_MIN_PATH_RAD);
    const bool ballistic = (k.peak_a >= REJ_MIN_PEAK_A);
    if (!rotational && !ballistic) {
        return (k.peak_w < REJ_MIN_PEAK_W) ? REJ_TOO_STILL : REJ_TOO_SHALLOW;
    }
}

// Impact, by spike WIDTH rather than magnitude.
//
// R3 change, forced by the sensor swap. With the MPU6050 at +/-8 g a 7.5 g
// table knock was separable from a 3 g cast by magnitude alone. The BNO055's
// fusion algorithm locks the accelerometer to +/-4 g, and gravity consumes one
// of them, so a knock and a hard Stupefy BOTH saturate near 3 g and magnitude
// can no longer tell them apart.
//
// Duration can, and always could: a collision reaches the spike level for one
// or two samples and is gone, whereas a thrust holds it for tens. A window is
// an impact if it reaches the spike level but never HOLDS it. This is a better
// test than the one it replaces, and it happens to survive clipping.
if (k.peak_a >= REJ_SPIKE_G && k.spike_run < REJ_SPIKE_MIN_RUN) return REJ_IMPACT;

return REJ_NONE;
}

```

Stages 2 and 3

firmware/wand_infer/reject.cpp

```

// Stages 2 and 3 -- the network's verdict, then confidence.
// -----
Decision judgeProbs(const float* probs, int n_classes) {
    Decision d{GESTURE_NONE, REJ_NONE, 0.f, 0.f, 3};

    int i1 = 0;
    for (int i = 1; i < n_classes; ++i) if (probs[i] > probs[i1]) i1 = i;
    int i2 = -1;
    for (int i = 0; i < n_classes; ++i)
        if (i != i1 && (i2 < 0 || probs[i] > probs[i2])) i2 = i;

    d.p1 = probs[i1];
    d.p2 = (i2 >= 0) ? probs[i2] : 0.f;

    // Stage 2
    if (i1 == GESTURE_NOISE) { d.reason = REJ_NN_NOISE; d.stage = 2; return d; }

    // Stage 3. Thresholds come from wand_thresholds.h and are MEASURED --
    // see that header for why hardcoding them is a mistake.
    if (probs[GESTURE_NOISE] >= REJ_TAU_NOISE) { d.reason = REJ_NOISE_PROB; return d; }
    if (d.p1 < REJ_TAU_CONF) { d.reason = REJ_LOW_CONF; return d; }
    if ((d.p1 - d.p2) < REJ_TAU_MARGIN) { d.reason = REJ_LOW_MARGIN; return d; }

    d.cls = (int8_t)i1;
    d.reason = REJ_NONE;
    return d;
}

```

8.3 Five defects in one subsystem, found and fixed before any data existed

Stage 1 is forty lines of comparisons and it has now been wrong five separate times. That is not a comment on the people who wrote it; it is what a threshold does when it is chosen by reasoning about a signal instead of measuring one. Each fix is below, in the order they were found.

Defect 1 — the bonus spell was unreachable (R2). The first plausibility gate applied a single peak-rotation floor and rejected **100% of Stupefy**, because Stupefy is defined as a thrust that barely rotates. The property that makes it a good bonus spell was exactly what the gate threw away, and the network never saw one. On the bench this looks like a model that cannot learn Stupefy.

The fix is a disjunction rather than a conjunction: a window is plausible if it is rotationally energetic *or* ballistically energetic. Scratching and pacing are neither, which is what keeps the pair separable.

Defect 2 — the impact test was unreachable (R3). With the MPU-6050 at ± 8 g, a 7.5 g table knock was separable from a 3 g cast by magnitude alone. The BNO055's fusion locks the accelerometer at ± 4 g, so a knock and a hard Stupefy *both* saturate near 3 g. In our bootstrap corpus **65% of Stupefy captures reach the ceiling**. Magnitude can no longer separate them.

Duration can, and always could. A collision reaches the spike level for one or two samples and is gone; a thrust holds it for tens. The test became: *a window is an impact if it reaches the spike level but never holds it*.

Defect 3 — the replacement test, chosen by intuition, rejected 9% of Stupefy. Having designed a better test we then set its two constants by feel (2.40 g, 4 samples) and made the same class of error a third time. Sweeping the pair moved it to 2.00 g, where it rejected no spells at all and caught five times as much noise. The intuitive setting had been worse on *both* axes simultaneously. That is the argument for sweeping rather than guessing, in one line.

Defect 4 — that sweep's grid stopped below the answer R4. The sweep ran levels from 1.6 g to 2.4 g, because R3 believed 3 g was the physical ceiling. Once §6.5 corrected the reportable range to 5 g, that ceiling turned out to be *inside* the search space rather than outside it — and a sweep whose grid stops short does not report that it stopped short. It reports a winner, with the same confident formatting as a sweep that searched the whole space. The grid now runs to 4.4 g and the selection rule takes a small budget of spell loss rather than demanding exactly zero, because with realistic lever-arm physics a gentle cast from a light-handed caster genuinely lands near the threshold and a hard zero-loss constraint just picks whichever corner of the grid happens to be degenerate.

spike level	min run	spell false-rejection	noise caught	
2.0 g	2	0.6%	15.6%	too narrow to catch a knock's ring-down
2.0 g	3	0.7%	49.6%	chosen
2.0 g	4	1.3%	45.2%	the R3 setting — now dominated
2.8 g	3	2.0%	55.2%	buys 6 points of noise for 3× the spell loss
4.4 g	2	0.0%	27.0%	the only <i>exactly</i> -zero-loss point, and much worse

Defect 5 — the energy gate was strictly dominated R4. The three energy floors had been set from per-feature percentiles, which is the wrong operation for a gate that is a *disjunction*: a Stupefy has almost no swept angle and does not need any, so taking the 1st percentile of swept angle over all spells drags the rotational floor down to a value chosen by the one class that never uses that arm. Sweeping the triple jointly, and evaluating the disjunction exactly as `reject.cpp` does, showed the R3 setting was not merely suboptimal but dominated:

peak ω	path	peak a	spell loss	noise stopped	
3.20	1.80	1.50	0.9%	16.7%	the sweep's argmax
2.20	1.00	1.80	2.0%	14.4%	the R3 setting — dominated by both neighbours
0.90	0.30	1.00	0.0%	11.1%	chosen

The chosen point is *not* the sweep's argmax, and that is a deliberate override worth defending. The entire gate is worth at most 17 points of noise, every point beyond about 11 of them costs real casts, and the two outcomes are not commensurate: a stopped noise window saves five milliseconds of inference that were available anyway, while a rejected spell is a lost point in front of a judge with no recourse. So the gate's job description was narrowed to the thing it is actually good at — deciding whether anything happened at all — and everything else was handed to Stage 2, which is a trained network with a real NOISE class and is far better at this than three scalars can be.

That narrowing exposed a gap in the corpus, incidentally: at 0.90/0.30/1.00 the gate initially stopped *nothing*, because every synthetic noise behaviour was energetic. The missing case was the one the gate exists for — the trigger pressed with nothing behind it, a fumbled button, a caster steadying themselves — so a physiological-tremor noise class was added, and the 11.1% is that class being caught.

Gate performance after all five fixes:

Class	Passes Stage 1	Rejection reasons
Alohomora, Wingardium Leviosa	100%	none
Lumos, Stupefy	99%	impact 1% — a grazing peak from the gentlest casters
Expelliarmus	98%	impact 2%
NOISE	39%	impact 50%, no motion 11% — 61% stopped before inference

THE PRINCIPLE THESE FIVE DEFECTS ESTABLISH

Stage 1 is a cheap pre-filter, not a classifier, and its job description is asymmetric. It must almost never reject a real spell, because a rejected spell is a lost point and the network never gets a say; any noise it happens to stop is a bonus that saves a few milliseconds. Its thresholds are therefore set from the *real spell* distribution and never from intuition about what noise looks like. Three corollaries the five defects add: sweep a gate *as it is evaluated*, not one feature at a time; make the grid wider than you think the answer can be, because a truncated sweep reports a winner rather than a warning; and when the sweep's argmax costs recall for a few points of noise, override it and say why.

`training/calibrate.py` prints the marginal percentiles for orientation, then sweeps both arms of the gate as the firmware actually evaluates them.

`training/calibrate.py`

```
def sweep_energy_gate(caps, budget: float = 0.01) -> None:
    """Choose REJ_MIN_PEAK_W, REJ_MIN_PATH_RAD and REJ_MIN_PEAK_A jointly.

    These three do not act independently -- the gate fires if a window is
    rotationally energetic (peak_w AND path_rad) OR ballistically energetic
    (peak_a) -- so setting each one from its own marginal percentile is the wrong
    operation. A Stupefy has almost no path_rad and does not need any; reading
    the p1 of path_rad over ALL spells therefore drags the rotational floor down
    to a value chosen by a class that never uses that arm.

    So sweep the triple, evaluate the disjunction exactly as reject.cpp does, and
```

```

take the point that stops the most noise subject to a small budget of spell
loss. The budget is not zero, deliberately: with the lever-arm physics
modelled, a gentle cast from a light-handed caster genuinely does land near
the floor, and a hard zero-loss constraint just picks whichever corner of the
grid happens to be degenerate.
"""
import wand_config as _C
S = [c.kin for c in caps if c.label != NOISE]
N = [c.kin for c in caps if c.label == NOISE]
if not S or not N:
    return

def passes(k, W, P, A):
    return (k.peak_w >= W and k.path_rad >= P) or (k.peak_a >= A)

print(f"\nStage 1 energy gate -- joint sweep (max noise stopped, "
      f"spell loss <= {budget*100:.0f}%):")
best = None
for W in (1.2, 1.6, 2.0, 2.4, 2.8, 3.2):
    for P in (0.4, 0.7, 1.0, 1.4, 1.8):
        for A in (1.2, 1.5, 1.8, 2.1, 2.5):
            loss = sum(1 for k in S if not passes(k, W, P, A)) / len(S)
            stop = sum(1 for k in N if not passes(k, W, P, A)) / len(N)
            if loss <= budget and (best is None or stop > best[4]):
                best = (W, P, A, loss, stop)

if best:
    W, P, A, loss, stop = best
    print(f"  choose REJ_MIN_PEAK_W = {W:.2f}, REJ_MIN_PATH_RAD = {P:.2f}, "
          f"REJ_MIN_PEAK_A = {A:.2f}")
    print(f"    spell loss {loss*100:.1f}%, noise stopped {stop*100:.1f}%")
    print(f"  currently {_C.REJ_MIN_PEAK_W:.2f} / {_C.REJ_MIN_PATH_RAD:.2f} "
          f"/ {_C.REJ_MIN_PEAK_A:.2f}")
else:
    print(f"  no point meets a {budget*100:.0f}% loss budget -- widen the grid")

```

training/calibrate.py

```

def sweep_spike_test(caps, budget: float = 0.01) -> None:
    """Choose REJ_SPIKE_G and REJ_SPIKE_MIN_RUN from data, on the same rule.

    The impact test is two-dimensional -- a level and a width -- so it cannot be
    read off a percentile at all. Sweep the pair and take the point that catches
    the most noise subject to the same small budget of spell false-rejection.

    Getting this wrong is not hypothetical, twice over. At the level and width
    first chosen by intuition (2.40 g, 4 samples) the test rejected 9% of Stupefy
    -- the bonus spell, whose whole signature is a large acceleration -- while
    catching only 4% of noise. And the sweep that replaced it originally stopped
    at 2.4 g, because R3 believed 3 g was the ceiling; once the lever-arm physics
    put real casts up at 5 g, that ceiling was inside the search space rather
    than outside it, and a sweep whose grid stops short does not report that it
    stopped short -- it reports a winner.
    """
    import wand_config as _C
    from preprocess import to_physical
    from dataset import _read_csv

    def peak_and_run(cap, level):
        raw, _, _ = _read_csv(cap.path)
        _, acc, _ = to_physical(raw)
        am = np.linalg.norm(acc, axis=1)
        run = best = 0
        for v in am:
            run = run + 1 if v >= level else 0
            best = max(best, run)
        return float(am.max()), best

    spells = [c for c in caps if c.label != NOISE]
    noise = [c for c in caps if c.label == NOISE]
    if not spells or not noise:

```

```

return

print(f"\nStage 1 impact test -- sweep (max noise caught, spell loss <= "
      f"{budget*100:.0f}%):")
print(f"{'spike_g':>9}{'min_run':>9}{'spell FALSE-REJ':>17}{'noise caught':>14}")
best = None
# The level range runs to 4.4 g because the reportable range is +/- 5 g per
# axis, not the 3 g R3 assumed. A sweep that stops at 2.4 -- as this one did
# before the geometry was modelled -- cannot see that the best operating
# point is above its own ceiling, and reports a "best" that is merely the
# least bad of the points it looked at.
for level in (1.8, 2.0, 2.4, 2.8, 3.2, 3.6, 4.0, 4.4):
    S = [peak_and_run(c, level) for c in spells]
    N = [peak_and_run(c, level) for c in noise]
    for width in (2, 3, 4, 6, 8):
        fr = sum(1 for p, b in S if p >= level and b < width) / len(S)
        ct = sum(1 for p, b in N if p >= level and b < width) / len(N)
        mark = ""
        if fr <= budget and (best is None or ct > best[3]):
            best = (level, width, fr, ct)
            mark = " <-- best so far"
        print(f"{level:9.1f}{width:9d}{fr*100:>16.1f}%{ct*100:>13.1f}%{mark}")
if best:
    print(f"\n choose REJ_SPIKE_G = {best[0]:.2f}, REJ_SPIKE_MIN_RUN = {best[1]}"
          f" (spell loss {best[2]*100:.1f}%, noise caught {best[3]*100:.1f}%)")
    print(f" currently {_C.REJ_SPIKE_G:.2f} / {_C.REJ_SPIKE_MIN_RUN}")

```

8.4 Thresholds are measured, not chosen

The confidence and margin thresholds started as intuition: 0.80 and 0.25. The diagnostic run that followed produced the most useful single result of the bring-up phase — a model that was **100% correct by argmax** while the confidence gate discarded more than half of its answers.

The cause is structural. With label smoothing at $\epsilon = 0.05$ over six classes, the training target for the correct class is $1 - \epsilon + \epsilon/K = 0.958$, and a converged model's honest top-1 probability sits well below that. On the R3 model the highest softmax value observed anywhere in the corpus was **0.660**. A threshold of 0.80 is therefore not conservative — it is unreachable, and it presents as a broken model.

The response was not to pick a better number by hand but to remove the number from the design. The three softmax thresholds now live in `shared/wand_thresholds.h`, generated by `training/calibrate.py` from the pooled leave-one-caster-out probability distribution, carrying a `WAND_THRESHOLDS_CALIBRATED` flag that a unit test checks. The selection rule encodes the scoring asymmetry: highest recall whose out-of-sample misfire rate is still zero, breaking ties toward the larger margin, because margin failures are the Lumos/Alohomora confusions and those are the ones a judge will reproduce deliberately.

`training/calibrate.py`

```
def operating_point(probs: np.ndarray, y: np.ndarray, budget: float):
    order = np.argsort(probs, axis=1)
    top1 = order[:, -1]
    p1 = probs[np.arange(len(probs)), order[:, -1]]
    p2 = probs[np.arange(len(probs)), order[:, -2]]
    pn = probs[:, NOISE]
    is_spell = y != NOISE

    best = None
    rows = []
    for tau in np.arange(0.30, 0.96, 0.025):
        for margin in np.arange(0.00, 0.61, 0.025):
            for tn in (0.35, 0.50):
                fire = (top1 != NOISE) & (p1 >= tau) & ((p1 - p2) >= margin) & (pn < tn)
                hit = fire & is_spell & (top1 == y)
                mis = fire & (~is_spell | (top1 != y))
                recall = hit.sum() / max(is_spell.sum(), 1)
                misfire = mis.sum() / max(len(y), 1)
                rows.append((tau, margin, tn, recall, misfire))
            if misfire <= budget:
                key = (recall, margin)
                if best is None or key > (best[3], best[1]):
                    best = (tau, margin, tn, recall, misfire)

    return best, rows
```

The measured effect

Configuration	τ	margin	cascade accuracy
Placeholder thresholds, guessed	0.55	0.25	0.452
Calibrated from the LOCO distribution	0.300	0.050	0.983

Same quantised model, same data, same cascade. The entire difference is three constants that were measured instead of guessed. The calibration run also reports the operating point it achieved — recall 1.000 at a misfire rate of 0.0000 — and stamps the generated header `SYNTHETIC CALIBRATION – rerun on real captures before the demo`, with the calibrated flag left at 0.

9 Validation methodology

9.1 Leave-one-caster-out, and why not a random split

A random split places repetitions from the same person on both sides of the line, so the model can memorise that person's idiosyncratic wrist signature and still look excellent. The judges are strangers. LOCO tests the only question that matters — does this work on a hand it has never seen — and it will read several points lower than a random split. That lower number is the one worth reporting, because it is the one that can be defended when a judge picks up the wand. A sixth caster's data is additionally held out entirely and never enters any training or calibration loop, as a check against the LOCO procedure itself having been tuned against.

9.2 The Python/C++ parity harness, and two negative controls

This is the most valuable test in the repository. Every other bug in this project announces itself: a wiring fault gives no I2C device, a bad build flag fails to compile, a bad model gives visibly wrong classes. A preprocessing divergence gives none of that. Both chains run clean, the training accuracy is excellent, the firmware is fast and stable, and the wand classifies at chance. There is nothing to grep for and no error to read.

THE SCHEDULE TRICK THAT MADE THIS AVAILABLE ON DAY ONE

This check is normally described as something you run once the hardware works and real gestures exist — which places the project's highest-risk defect at the *end* of the timeline. But `preprocess.cpp` includes nothing from Arduino: only `stdint.h` and `math.h`. So it compiles and runs on a laptop, today, against any CSV. The divergence is ruled out before the sensor is even wired.

```
g++ -O2 -std=c++17 -I../shared -I../common -o parity_host \
    parity_host.cpp preprocess.cpp reject.cpp

checked 60 captures
  worst tensor element difference : 6.321e-06   (synth00_LUMOS_5.csv)
  worst kinematic difference      : 1.949e-06
  Stage 1 gate verdict mismatches : 0
  tolerance                      : 1.0e-03
PARITY OK -- preprocess.py/.cpp and the Stage 1 gate all agree.
```

The agreement is at the level of float32 rounding — two orders of magnitude inside the tolerance — which is what it should be, given Python computes in float64 and the firmware in float32.

A test that has never failed is not yet known to work

Three negative controls were introduced deliberately, each the most likely real-world version of a mistake this design invites. Two were caught immediately. **The third was not**, and that is the more useful result:

Injected defect	Divergence	× tol	Outcome
Quaternion multiply order reversed <code>qMul(q, q0c)</code> instead of <code>qMul(q0c, q)</code>	2.23×10^{-1}	223×	Caught. Quaternion multiplication does not commute, and both orders compile and run — the single most likely typo in this design.
Hemisphere continuity pass removed	1.92	1,920×	Caught. The magnitude is itself diagnostic: very close to 2.0, which is a full sign inversion of a channel spanning [-1, 1].
<code>REJ_SPIKE_MIN_RUN</code> off by one in the C++ header only	0	—	NOT caught. Passed clean. See below.

What the third control exposed. The harness compared the tensor and the kinematic features, and agreed on every number — correctly, because `spike_run` is computed against `REJ_SPIKE_G` and `REJ_SPIKE_MIN_RUN` is consulted only by `gateKinematics()`, which nothing was exercising. A test that covers the inputs to a decision but not the decision leaves the whole of Stage 1 free to diverge silently — the exact failure class the harness exists to eliminate, reproduced inside the harness itself.

The fix was to make `parity_host` link `reject.cpp` (which, like `preprocess.cpp`, includes nothing from Arduino) and emit the gate verdict as a raw enum value, compared against the Python mirror. Integers rather than names, because the C++ names are for an OLED ("NO ARC") and the Python side names the condition ("TOO_SHALLOW") — a comparison that is insensitive to wording and sensitive to behaviour. Re-run against the same injected defect, the harness now reports `gate verdict mismatches : 1` and names the file.

All three were reverted and parity re-confirmed. The R2 equivalent — a complementary filter seeded from zero rather than from the accelerometer — was caught at 7.0×10^{-2} ; that step no longer exists to get wrong.

9.3 Constant and layout parity

The second silent failure class is a constant drifting between the C++ header and the Python mirror. `test_config_parity.py` parses the headers and asserts every shared value, the class enumeration, and the derived constants. Three assertions are specific to R3: that the twelve channels tile the tensor exactly once with no gap or overlap; that NOISE remains the *last* class, since inserting it in the middle would silently relabel every gesture recorded before the change; and that `NORM_ACC_LIN` never exceeds what the fusion's locked ± 4 g range can physically report.

```
$ python -m pytest training/test_config_parity.py -q
.....S
10 passed, 1 skipped in 0.02s
```

The skip is deliberate and informative: it is the assertion that the demo build is not shipping uncalibrated thresholds, and it remains a skip until `calibrate.py` has run against real captures.

9.4 The synthetic quarantine

The bootstrap generator is what allowed the entire pipeline — tensor shapes, quaternion anchoring, augmentation, the network graph, int8 conversion, the C array export, arena sizing, the parity harness, and the threshold calibration path — to be built and proven before any gesture existed.

It is not a dataset, and three independent locks enforce that: output goes only to `data/synthetic/` and the generator refuses any path containing `raw`; every file carries a `# SYNTHETIC` first line and every row a `synthetic=1` column; and the loader refuses to run unless `data/raw` holds at least 50 genuine captures, or `--allow-synthetic` is passed explicitly — in which case every printed number is prefixed `SYNTHETIC -- NOT A RESULT`.

```

def load_dataset(root: str | Path = C.REAL_DATA_DIR,
                 allow_synthetic: bool = False) -> list[Capture]:
    """Load every capture under `root`.

    Raises unless either (a) root holds at least MIN_REAL_CSV_FILES genuine
    captures, or (b) allow_synthetic is explicitly True.
    """
    root = Path(root)
    n_real = _count_real_csv(root)

    if not allow_synthetic and n_real < C.MIN_REAL_CSV_FILES:
        raise RuntimeError(
            f"No real dataset present -- refusing to run on synthetic data.\n"
            f" {root} holds {n_real} genuine captures, need >= {C.MIN_REAL_CSV_FILES}.\n"
            f" Collect data with training/collect.py, or pass --allow-synthetic\n"
            f" if you are only smoke-testing the pipeline (results will be\n"
            f" stamped SYNTHETIC and must never enter the report).")

    out: list[Capture] = []
    for path in sorted(root.rglob("*.csv")):
        raw, label, caster, synthetic = _read_csv(path)
        if label < 0 or len(raw) < C.MIN_GESTURE_SAMPLES:
            continue
        if len(raw) > C.MAX_GESTURE_SAMPLES + C.PREROLL_SAMPLES + C.POSTROLL_SAMPLES:
            raw = raw[: C.MAX_GESTURE_SAMPLES + C.PREROLL_SAMPLES + C.POSTROLL_SAMPLES]
        tensor, kin = preprocess(raw)
        out.append(Capture(tensor, kin, label, caster, path, synthetic))

    if not out:
        raise RuntimeError(f"{root} contained no usable captures")
    return out

```

The bootstrap corpus produces argmax accuracy of 0.999 ± 0.003 across the six held-out folds. That figure is reported here *only* as evidence that generated data is trivially separable and therefore worthless as a result. It is the same signature — near-perfect scores, near-zero variance — that this project has already been misled by once.

9.5 Making the generator obey the wand's physics, and auditing that it does R4

Better physics does not make a synthetic corpus into evidence. What it makes it into is a usable *engineering target*: the tensor ranges, the saturation rate, the gate pass rates and the arena size become the ones the real wand will produce, so every constant sized against them is sized against something real. Everything in §6.5 and §8.3 came out of that change, and none of it was reachable before.

What the caster commands, and what is derived

Each spell is now specified as a sequence of *angular excursions in radians* and *hand displacements in metres*, over named fractions of the gesture duration. There is not a single acceleration constant anywhere in the spell definitions, and there must never be one: the moment somebody adds one, the accelerometer channel stops being a consequence of the gesture and goes back to being a free parameter.

```
def _wingardium(t, T, g, m: Move):
    """Swish and flick. A broad lateral yaw sweep driven from the elbow, then a
    crisp upward pitch from the wrist. The two sub-moves are ordered, and that
    ordering is the reason the classifier flattens the temporal axis instead of
    global-average-pooling it."""
    m.turn(AX_YAW, g(1.55), 0.05 * T, 0.50 * T)          # swish
    m.reach([0.0, g(0.20), 0.0], 0.05 * T, 0.50 * T)
    m.turn(AX_PITCH, -g(0.95), 0.60 * T, 0.30 * T)      # flick
    m.reach([0.0, 0.0, g(0.06)], 0.60 * T, 0.30 * T)
    return m, 0.35 * C.PIVOT_WRIST_M + 0.65 * C.PIVOT_ELBOW_M
```

Both commanded quantities use the **minimum-jerk** profile $s(u) = 10u^3 - 15u^4 + 6u^5$ (Flash & Hogan, 1985), the standard model of human point-to-point limb movement. It is not chosen for elegance: its first derivative is zero at both ends, and so is its *second*. A profile with non-zero endpoint acceleration would inject a step into α , and therefore a discontinuity into the accelerometer channel, that no real arm produces and that 100 Hz sampling would alias. Specifying rotations as excursions also means the rate profile integrates to *exactly* the requested angle, which is a quantity that can be checked against a video of someone casting — as a hand-picked amplitude in counts cannot.

training/synth_bootstrap.py

```
def minjerk_rate(u: np.ndarray) -> np.ndarray:
    """d/du of minjerk_s: 30 u^2 (1-u)^2, zero outside [0, 1].

    Unit area over the interval, so multiplying by (excursion / duration) gives
    a rate profile that integrates to EXACTLY the requested excursion. Peak
    value is 1.875, at u = 0.5.
    """
    inside = (u >= 0.0) & (u <= 1.0)
    uc = np.clip(u, 0.0, 1.0)
    return np.where(inside, 30.0 * uc ** 2 * (1.0 - uc) ** 2, 0.0)
```

Everything the sensor sees then follows from the rigid-body relation of §4.2. Note where the ± 4 g clip is applied — to the raw signal, *before* gravity is subtracted, because that is where the chip applies it. Clipping the linear channel directly would be wrong by up to 1 g and would understate how often a hard cast actually saturates.

training/synth_bootstrap.py

```
# generation
# =====
def synth_capture(label: int, caster_seed: int,
                  rng: np.random.Generator) -> np.ndarray:
    """Return an (N, 10) int16 array in exactly the layout the logger emits:
    qw qx qy qz | lax lay laz | gx gy gz."""
    cr = np.random.default_rng(caster_seed)

    # Per-caster style. These are the axes along which real casters differ, and
    # the leave-one-caster-out split is only meaningful because they exist.
    gain = cr.uniform(0.82, 1.22)          # how vigorously this person casts
    dur_bias = cr.uniform(0.86, 1.16)      # how fast
    hold = cr.uniform(-0.40, 0.40, 3)      # the pose the wand is held in
    pivot_scale = cr.uniform(0.80, 1.25)   # arm length / wrist-vs-elbow habit
    gyro_bias = cr.normal(0.0, 0.020, 3)   # residual fusion bias, rad/s

    def g(x):
        """Per-rep excursion scaling: caster style times within-caster variation."""
        return x * gain * rng.uniform(0.90, 1.10)

    pre, post = C.PREROLL_SAMPLES, C.POSTROLL_SAMPLES

    if label == int(C.WandGestureClass.NOISE):
        kind = NOISE_KINDS[rng.choice(len(NOISE_KINDS), p=NOISE_WEIGHTS)]
        T = float(rng.uniform(0.65, 1.95))
        n_g = int(round(T * C.FS_HZ))
```



```

    t = np.arange(n_g) * C.DT_S
    mv, pivot = kind(t, T, rng, Move(t, C.DT_S))
    mv.w *= gain
else:
    fn, (lo, hi) = SPELLS[C.WandGestureClass(label)]
    T = float(np.clip(rng.uniform(lo, hi) * dur_bias, 0.45, 1.95))
    n_g = int(round(T * C.FS_HZ))
    t = np.arange(n_g) * C.DT_S
    mv, pivot = fn(t, T, g, Move(t, C.DT_S))

# --- pre-roll and post-roll -----
# The rate pads with zeros (the caster is still), but the hand PATH pads by
# HOLDING its endpoint value -- padding a position with zeros would inject a
# step, and the second derivative of a step is an impulse that would appear
# as a phantom impact at the seam of every single capture.
w = np.vstack([np.zeros((pre, 3)), mv.w, np.zeros((post, 3))])
p = np.vstack([np.repeat(mv.p[:1], pre, axis=0), mv.p,
               np.repeat(mv.p[-1:], post, axis=0)])
imp = np.vstack([np.zeros((pre, 3)), mv.impulse, np.zeros((post, 3))])
n = len(w)

# --- rigid-body kinematics -----
# r is the fixed vector from the pivot to the sensor, in the BODY frame:
# along the tube by (pivot + 37 cm), plus the 12 mm the PCB stands off the
# tube's centreline. The radial offset is small but it is what makes a pure
# roll about the wand's own axis visible at all -- with r exactly on the
# axis, roll would produce no acceleration whatsoever.
lever = pivot * pivot_scale + C.SENSOR_FROM_BUTT_M
r = np.array([lever, 0.0, C.SENSOR_RADIAL_OFF_M])

alpha = ddt(w, C.DT_S)                    # rad/s^2, body frame
a_lever = cross(alpha, r) + cross(w, cross(w, r)) # m/s^2, body frame

R = rot_from_omega(w, C.DT_S)              # body -> anchor
Rhold = euler_matrix(*hold)                # anchor -> world
R = np.einsum('ij,njk->nik', Rhold, R)    # body -> world

a_pivot_anchor = d2dt2(p, C.DT_S) + imp    # m/s^2, anchor frame
a_pivot_world = np.einsum('ij,nj->ni', Rhold, a_pivot_anchor)

# a_sensor = a_pivot + alpha x r + omega x (omega x r), in the body frame.
lin_ms2 = np.einsum('nji,nj->ni', R, a_pivot_world) + a_lever
lin_body = lin_ms2 / C.G_MS2               # -> g

# --- gravity and the +/-4 g clip -----
# Gravity is +1 g "up" in the world frame. The raw accelerometer measures
# imparted acceleration PLUS the gravity reaction in its own frame; the
# fusion subtracts its gravity estimate and reports the remainder as LIA.
# The clipping happens on the RAW signal, before that subtraction, and
# reproducing it in the right place matters: clipping the linear channel
# directly would be wrong by up to 1 g and would understate how often a hard
# cast actually saturates.
g_world = np.array([0.0, 0.0, 1.0])
g_body = np.einsum('nji,j->ni', R, g_world)
raw_body = np.clip(lin_body + g_body,
                   -C.BNO_FUSION_ACC_RANGE_G, C.BNO_FUSION_ACC_RANGE_G)
lin_body = raw_body - g_body

# --- sensor imperfection -----
# Smaller than a bare MPU6050's, because the fusion has already suppressed
# most of the white noise; what is left is mostly its own estimation error.
lin_body += rng.normal(0.0, 0.008, lin_body.shape)
w = w + gyro_bias * 0.25 + rng.normal(0.0, 0.004, w.shape)

quat = quat_from_matrix(R)
dq = rng.normal(0.0, 0.0015, (n, 3))
quat[:, 1:4] += dq
quat /= np.linalg.norm(quat, axis=1, keepdims=True)

# --- BNO055 behaviours the downstream code must survive -----
quat[rng.random(n) < 0.04] *= -1.0      # q and -q are the same rotation
quat[rng.random(n) < 0.004] = 0.0      # I2C hiccup -> all-zero frame

```

```

raw = np.empty((n, 10), dtype=np.int16)
raw[:, 0:4] = np.clip(quat * C.BNO_LSB_PER_QUAT, -32768, 32767).astype(np.int16)
raw[:, 4:7] = np.clip(lin_body * C.LINACC_LSB_PER_G, -32768, 32767).astype(np.int16)
raw[:, 7:10] = np.clip((w / C.DEG2RAD) * C.BNO_LSB_PER_DPS,
                        -32768, 32767).astype(np.int16)

for i in range(1, n):
    if rng.random() < 0.02:
        raw[i] = raw[i - 1]
return raw

```

One detail in there is worth pointing at, because getting it wrong produces a defect that looks exactly like a hardware fault. The pre-roll and post-roll pad the angular *rate* with zeros but pad the hand *path* by holding its endpoint value. Padding a position with zeros injects a step, and the second derivative of a step is an impulse — so every single capture would have carried a phantom impact at the seam, in exactly the shape the Stage 1 impact test is built to detect.

The audit: physics is checkable, and "it looks about right in a plot" is not a check

`training/audit_corpus.py` reduces a corpus to the quantities that can be checked against arithmetic and a stopwatch, and states a pass or a warning against a band for each. Every band is a property of a person waving a 50 cm stick rather than of this project's model, so **the same script is the acceptance test for the real corpus** once it exists. If the real numbers fall outside these bands, the bands were what was wrong, and the synthetic corpus tuned to them has to be regenerated before it is used to size anything.

```

SYNTHETIC -- NOT A RESULT | corpus audit: data/synthetic (810 captures)
geometry: wand 50 cm, BNO055 13 cm below the tip, +X towards the tip
lever arm to the sensor: 0.45 m (wrist) .. 0.99 m (shoulder)

--- kinematics, median [5th, 95th percentile] ---
class      n      peak |w| rad/s      peak |a| g      tip m/s      dur s      path      spike      clip%
LUMOS      108     4.56 [3.23, 6.39]  3.29 [1.85,5.84]  2.28 [1.62]  1.30      2.12      12      0.7%
ALOHOMORA  108     7.22 [5.24,10.53]  4.96 [3.33,5.72]  3.61 [2.62]  1.40      2.67      36      9.1%
EXPELLIARMUS 108     3.48 [2.92, 4.12]  2.05 [1.64,2.73]  1.74 [1.46]  1.64      2.06       7      0.0%
WINGARDIUM LEVIOSA 108     5.71 [4.29, 8.80]  3.89 [2.50,5.90]  2.86 [2.14]  1.29      2.36      26      4.8%
STUPEFY    108     1.94 [1.41, 2.61]  2.84 [1.70,4.36]  0.97 [0.70]  0.98      0.28       7      0.6%
NOISE      270     2.09 [0.26, 6.31]  4.34 [0.45,5.97]  1.04 [0.13]  1.60      0.89       2      1.1%

--- lever-arm coupling: corr(peak |w|, peak |a|) within each class ---
LUMOS +0.88  ALOHOMORA +0.89  EXPELLIARMUS +0.63
WINGARDIUM +0.93  STUPEFY +0.92  NOISE +0.35

--- tensor occupancy per channel group (fraction of range used) ---
accel (anchored)  p50 0.009  p99 0.751  max 1.000  at-rail 0.08%
gyro (anchored)  p50 0.001  p99 0.441  max 0.921  at-rail 0.00%
quaternion        p50 0.057  p99 1.000  max 1.000  at-rail 9.38%
|a|                p50 0.093  p99 0.962  max 1.000  at-rail 0.59%
|w|                p50 0.021  p99 0.569  max 0.978  at-rail 0.00%

all classes fall inside their physical plausibility bands.

```

The lever-arm coupling row is the one that only holds if the physics is real. A wand rotating about a pivot half a metre away *must* show acceleration that tracks angular rate; if peak $|a|$ and peak $|\omega|$ were uncorrelated within a spell class, the accelerometer channel would not be coming from the geometry — which is precisely the defect this revision exists to remove, so it is checked rather than assumed. Stupefy is hand-driven by construction and is exempted; NOISE is a mixture of unrelated behaviours and its low ρ is expected.

The audit also recovers something the CSV does not contain. The logger stores the fusion's gravity-free linear acceleration, so the raw accelerometer signal — and therefore the saturation rate — is not in the file. But the same file carries the orientation the fusion used to remove gravity, so it can be rebuilt:

```
def reconstruct_gravity(quat: np.ndarray) -> np.ndarray:
    """Body-frame gravity, in g, recovered from the orientation quaternion.

    The logger stores the fusion's gravity-free linear acceleration, so the raw
    accelerometer signal is not in the file -- but it can be rebuilt, because
    the same file carries the orientation the fusion used to remove gravity:

        g_body = R^T * (0, 0, 1) = rotate(conj(q), world_up)

    That makes the +/-4 g saturation rate measurable from a CSV alone, which
    matters because saturation is invisible in the linear channel: a clipped
    sample looks like an ordinary large number there, not like a rail.
    """
    up = np.zeros_like(quat[:, 1:4])
    up[:, 2] = 1.0
    return quat_rotate(quat_conj(quat), up)
```

This matters because saturation is otherwise invisible: a clipped sample looks like an ordinary large number in the linear channel, not like a rail. Alohomora saturates on 9.1% of its samples and Wingardium Leviosa on 4.8% — both expected for a 50 cm lever, both fine, and neither knowable without this reconstruction.

Two further BNO055 behaviours are reproduced on purpose, because each has a code path that must survive it: quaternion sign flips (4% of frames), occasional all-zero quaternion frames from an I2C hiccup (0.4%), and duplicate fusion frames from the beat between the chip's 100 Hz clock and the ESP32's (2%).

WHAT THIS SECTION DOES AND DOES NOT CLAIM

It claims the generator produces signals of the right shape and scale for the wand that was built, and that the constants sized against them are therefore sized against something real. It claims nothing whatsoever about whether the classifier works. The 0.999 in §9.4 remains fiction, and better physics makes it *more* convincing fiction, not less. Only real captures can produce an accuracy figure, and §13 is the plan to get them.

10 Firmware implementation walkthrough

10.1 One driver, three firmwares R4

The BNO055 driver lives in `firmware/common/bno055_fusion.cpp` and is linked by all three targets: the demo firmware, the logger that produces the training data, and the bring-up suite that decides whether the sensor is trustworthy at all. That sharing is the point rather than tidiness. A bring-up tool that configures the chip its own way tests a configuration that will never be flashed again, and the acceptance it grants is worthless; a logger that configures it differently from the demo firmware produces training data from a sensor the demo will not be using.

THE HAZARD THIS CONSOLIDATION REMOVED

The tree had accumulated **two incompatible BNO055 implementations**: the fusion driver, and a second one running the chip as a plain 6-axis sensor in ACCGYRO mode with its own six-field `RawSample` — same struct name, different fields, different units. Neither was reachable from a build, because `build_src_filter` happened to exclude both directories, so nothing failed and nothing warned. The moment anyone changed a filter, the firmware would have filled a fusion-shaped struct from raw accelerometer registers and classified at chance, with no error anywhere. The non-fusion path is deleted; its two genuinely good ideas — named `InitResult` failure causes rather than a bool, and read-back verification on every configuration write — were kept and are visible in the code below.

10.2 Sensor bring-up, and an assumption converted into a check

`firmware/common/bno055_fusion.cpp`

```
InitResult begin() {
    Wire.begin(PIN_IMU_SDA, PIN_IMU_SCL, I2C_FREQ_IMU);
    // The BNO055 stretches the I2C clock hard. A short timeout turns that into an
    // intermittent bus hang that passes every quick bench test and dies mid-demo.
    Wire.setTimeout(I2C_TIMEOUT_MS);

    // The chip does not answer I2C at all for ~650 ms after power-on. Polling
    // rather than a blind delay, because a bare delay hides the case where the
    // sensor is genuinely absent -- you want that to fail loudly at boot.
    const uint32_t deadline = millis() + BNO_POWERON_DELAY_MS + 800;
    bool answered = false;
    while ((int32_t)(millis() - deadline) < 0) {
        Wire.beginTransaction(I2C_ADDR_BNO);
        if (Wire.endTransmission() == 0) { answered = true; break; }
        delay(20);
    }
    if (!answered) return IMU_NO_DEVICE;

    // Give it the rest of the power-on window to start answering with a sane ID.
    for (int i = 0; i < 40; ++i) {
        s_chip_id = r8(BNO_REG_CHIP_ID);
        if (s_chip_id == BNO_CHIP_ID_VALUE) break;
        delay(20);
    }
    if (s_chip_id != BNO_CHIP_ID_VALUE) return IMU_BAD_CHIP_ID;

    // FORCE PAGE 0 FIRST. The BNO055 has two register pages, and UNIT_SEL (0x3B)
    // and OPR_MODE (0x3D) are page-0 addresses. If the chip is left on page 1 --
    // by an earlier experiment, or by a reset partway through someone else's
    // configuration sequence -- both writes below land on page-1 addresses, are
    // discarded, and the wand runs in whatever mode and units it was left in with
    // nothing anywhere reporting a problem. One register write closes it.
    if (!w8_verify(BNO_REG_PAGE_ID, 0x00)) return IMU_PAGE_FAILED;

    w8(BNO_REG_OPR_MODE, BNO_MODE_CONFIG);
```

```

delay(BNO_MODE_SWITCH_MS);          // >= 19 ms leaving an operating mode
w8(BNO_REG_SYS_TRIGGER, 0x00);      // internal oscillator
w8(BNO_REG_PWR_MODE, 0x00);         // normal power
delay(10);

// Units written explicitly even though 0x00 is the power-on default, and
// VERIFIED rather than assumed: a chip left in mg or rad/s by an earlier
// experiment would silently rescale every capture by 2% (m/s^2 vs mg) or by
// 57x (dps vs rad/s), and nothing downstream would report an error.
if (!w8_verify(BNO_REG_UNIT_SEL, BNO_UNIT_SEL_VALUE)) return IMU_UNIT_FAILED;

w8(BNO_REG_OPR_MODE, BNO_MODE_IMU);  // accel + gyro fusion, magnetometer OFF
delay(BNO_MODE_SWITCH_MS);          // >= 7 ms entering an operating mode
if ((r8(BNO_REG_OPR_MODE) & 0x0F) != BNO_MODE_IMU) return IMU_MODE_FAILED;

s_errors = 0;
return IMU_OK;
}

```

A SILENT-FAILURE MODE FOUND LATE, FROM A PARALLEL IMPLEMENTATION

The BNO055 has **two register pages**, selected by `PAGE_ID` at 0x07. This design never writes a page-1 register — in fusion mode the algorithm owns the ranges — so it initially looked like a non-issue. It is not: `UNIT_SEL` (0x3B) and `OPR_MODE` (0x3D) are *page-0* addresses. A chip left on page 1 by an earlier experiment, or by a reset partway through someone else's configuration sequence, would silently discard both writes, and the wand would run in whatever mode and units it happened to be left in — with the sensor streaming perfectly plausible numbers and nothing anywhere reporting a problem. One register write closes it, and every configuration write is now read back and verified rather than assumed. Claiming to guard against a dropped write without reading it back is not a guard.

Two further details are deliberate. The power-on wait is a *poll* rather than a blind delay: the chip does not answer I2C at all for about 650 ms, but a bare delay would also hide the case where the sensor is genuinely absent, and that should fail loudly at boot. And `UNIT_SEL` is written even though 0x00 is its power-on default, because a chip left in mg or rad/s by an earlier experiment would silently rescale every capture with nothing downstream reporting an error.

The datasheet states that the quaternion registers are valid in every fusion mode. "The datasheet says" is exactly the kind of claim this project checks rather than trusts, because if it were wrong the failure would be silent — the quaternion would read zero, the preprocessing would substitute the identity, four of twelve channels would be constant, and the wand would simply classify worse:

firmware/common/bno055_fusion.cpp

```

bool quaternionIsLive() {
    RawSample s{};
    for (int attempt = 0; attempt < 20; ++attempt) {
        if (read(&s)) {
            const float n = sqrtf((float)s.qw * s.qw + (float)s.qx * s.qx +
                                   (float)s.qy * s.qy + (float)s.qz * s.qz);
            if (n > 0.8f * BNO_LSB_PER_QUAT && n < 1.2f * BNO_LSB_PER_QUAT) return true;
        }
        delay(25);
    }
    return false;
}

```

10.3 The TFLM operator resolver and inference

firmware/wand_infer/wand_infer.cpp

```

bool begin() {
    const tflite::Model* model = tflite::GetModel(g_wand_model);
    if (model->version() != TFLITE_SCHEMA_VERSION) return false;

    static tflite::MicroMutableOpResolver<7> resolver;
    resolver.AddConv2D();
    resolver.AddDepthwiseConv2D();
    resolver.AddFullyConnected();
    resolver.AddMaxPool2D();
    resolver.AddReshape();
    resolver.AddSoftmax();
    resolver.AddRelu();

    static tflite::MicroInterpreter interp(model, resolver, s_arena, sizeof(s_arena));
    if (interp.AllocateTensors() != kTfLiteOk) return false;

    s_interp = &interp;
    s_in = interp.input(0);
    s_out = interp.output(0);
    // If the model was retrained with a different channel count and the firmware
    // was not rebuilt, this is where it must fail -- loudly, at boot.
    if (s_in->bytes != (size_t)TENSOR_LEN) return false;
    s_in_scale = s_in->params.scale; s_in_zp = s_in->params.zero_point;
    s_out_scale = s_out->params.scale; s_out_zp = s_out->params.zero_point;
    return true;
}

```

A `MicroMutableOpResolver` listing exactly the seven kernels this graph uses, rather than `AllOpsResolver` — the latter links every kernel TFLM has, well over 100 KB of flash for operations this model will never execute. Naming them also means an accidental architecture change that introduces a new operator fails loudly at `AllocateTensors()` instead of silently bloating the binary. The added `s_in->bytes` check is an R3 guard: a model retrained with a different channel count against firmware that was not rebuilt would otherwise read past its input tensor.

10.4 A window of stale data that passed the duration gate R4

The ring buffer holds 320 samples — 3.2 seconds. The capture window holds 230. Hold the trigger for six seconds and the requested window is 600 samples long, of which the first 280 have been overwritten.

The original code clamped the *count* to the window capacity and then read from the original start position. So it returned 230 samples of unrelated history, spliced across the ring's write pointer, carrying a duration of exactly 2.30 s — which is `REJ_MAX_DURATION_S`, and the bound is a strict `>`. The window passed the duration gate and went to the network. A garbage window that gets *rejected* is a non-event; a garbage window that gets *classified* is a spell nobody cast, in front of a judge, from a wand that was simply being held.

```
firmware/wand_infer/wand_infer.cpp
```

```

// Copies `count` samples starting at sequence number `start_seq` into s_window.
// Returns the number copied, or 0 if the request cannot be satisfied.
//
// R4 FIX -- an over-long hold used to produce a plausible window of stale data.
// The old version clamped `count` to the window capacity (230) but kept reading
// from the ORIGINAL start position. Hold the trigger for six seconds and that
// position is 600 samples back in a 320-sample ring: long since overwritten. The
// result was 230 samples of unrelated history, spliced across the ring's write
// pointer, carrying a duration of exactly 2.30 s -- which is REJ_MAX_DURATION_S,
// and the bound is `>`, so it passed the duration gate and went to the network.
// A garbage window that is rejected is a non-event; a garbage window that is
// CLASSIFIED is a spell nobody cast.
//
// Two changes. The caller now refuses an over-long hold outright, and this
// function additionally refuses to read data the ring can no longer vouch for.
// Belt and braces, because the two failures look identical from the outside and
// only one of them announces itself.
static int snapshot(int start_seq, int count) {
    if (count < 1) return 0;
    if (count > WINDOW_CAP) return 0; // caller must have caught this
    if ((int)s_seq - start_seq > RING_LEN) return 0; // already overwritten
    int idx = ((start_seq % RING_LEN) + RING_LEN) % RING_LEN;
    for (int i = 0; i < count; ++i) {
        s_window[i] = s_ring[idx];
        idx = (idx + 1) % RING_LEN;
    }
    return count;
}

```

Two changes, deliberately overlapping. The caller now rejects an over-long hold where the true held length is still known — once the count has been clamped, that information is gone and every over-long hold looks like an ordinary 2.3-second gesture. And the function independently refuses to read data the ring can no longer vouch for. The two failures look identical from outside and only one of them announces itself, which is the argument for having both.

10.5 Compiling the firmware without the toolchain R4

The ESP32 toolchain lives on one laptop. Everyone else's round trip for a typo was: push, ask, wait, read the error, fix, repeat. `tools/hostcheck/check.sh` type-checks all six firmware translation units with `g++ -fsyntax-only` against stubbed `Arduino.h`, `Wire.h`, the two Adafruit headers, `FastLED`, and the slice of TFLite Micro that `wand_infer.cpp` actually calls. The TFLM stubs mirror the real signatures, so a call that no longer matches the library fails here rather than on the ESP32.

```

$ ./tools/hostcheck/check.sh
host type-check (stubbed Arduino; not a functional test)
firmware/common/bno055_fusion.cpp          OK
firmware/wand_infer/preprocess.cpp          OK
firmware/wand_infer/reject.cpp              OK
firmware/wand_infer/parity_host.cpp         OK
firmware/wand_logger/wand_logger.cpp        OK
firmware/wand_tools/bno_bringup.cpp         OK
firmware/wand_infer/wand_infer.cpp          OK

```

It is worth being precise about what this is not. The stubs compute nothing; nothing here is executed, and a green result says only that the code compiles. It removes the flash-cycle-per-typo loop, which on a five-person team with one toolchain is most of the round trips, and it is not a substitute for `pio run`.

10.5 Graceful degradation and fault signalling

The usual SSD1306 example code responds to `begin()` returning false with `for(;;)`. If the OLED is miswired that makes the entire wand appear dead — including the LED ring, which would otherwise still be perfectly capable of casting.

The firmware records the failure and carries on. Conversely, sensor, quaternion and model failures *are* fatal, and each signals a distinct LED pattern so a fault can be diagnosed across a room with no serial port available:

Pattern	Meaning	First thing to check
Red, 300 ms blink	BNO055 not found, or would not enter fusion mode	SDA/SCL on GPIO21/22, address 0x28 not 0x68, 3.3 V present
Yellow, 150 ms blink	Sensor answers, but the quaternion is not live	<code>OPR_MODE</code> actually took; the chip may have fallen back to CONFIG
Magenta, 120 ms blink	Model or arena failure	Channel count mismatch between <code>model_data.cc</code> and the firmware build
Amber, steady	Waiting for gyro calibration	Nothing — hold the wand still for a few seconds

10.6 Build targets

platformio.ini

```
[env]
platform = espressif32
board = esp32dev           ; classic 30-pin ESP32-WROOM-32 DevKit
framework = arduino
monitor_speed = 921600
upload_speed = 921600
build_flags = -I shared -I firmware/common -Os
lib_deps =
  adafruit/Adafruit SSD1306 @ ^2.5.9
  adafruit/Adafruit GFX Library @ ^1.11.9
  fastled/FastLED @ ^3.6.0

; Versions pinned deliberately. Two teammates on different Adafruit_GFX minor
; versions produce boards that behave differently for no visible reason.
;
; Note the absence of Adafruit_BNO055. The driver is firmware/common/bno055_fusion.cpp
; and the register values are part of the frozen contract with the training data,
; so they belong somewhere training/test_config_parity.py can read them -- not
; inside a library call.
;
; R4: there is now exactly ONE driver, and all three firmware targets link it.
; The tree previously carried a second, incompatible BNO055 implementation that
; ran the chip as a plain 6-axis sensor in ACCGYRO mode with its own six-field
; RawSample. Both defined the same struct name; whichever was compiled first won.
; Neither was reachable from a build, so nothing failed -- until someone changed a
; build_src_filter, at which point the wand would have filled a fusion-shaped
; struct from raw registers and classified at chance with no error anywhere.

; --- BNO055 acceptance suite. Run this before soldering and before collecting.
[env:bringup]
build_src_filter = +<wand_tools/*> +<common/*>
build_flags =
  ${env.build_flags}
  -DDEBUG_SERIAL=1
```

`DEBUG_SERIAL` is the project's defence against the offloading rule, and a defence that depends on someone remembering to edit a `#define` before the demo is not a defence. Here it is a property of the selected build target, visible in the status bar and impossible to get wrong by forgetting. Library versions are pinned deliberately: two team members on different Adafruit_GFX minor versions produce boards that behave differently for no visible reason.

11 Latency and memory budget

11.1 The Quick-Draw path

Stage	Time	Basis
Trigger release debounce	20 ms	Design constant; 20 ms software on top of a ~4.5 ms hardware RC
Post-roll accumulation	100 ms	10 samples at 100 Hz — a design choice, see below
Preprocessing (230 samples)	~0.6 ms	Estimated. Down from ~2.5 ms in R2: the chain no longer contains a complementary filter or a gravity projection, so there are no transcendental calls left except two <code>sqrtof</code> per sample.
TFLM inference	~2–5 ms	Estimated: ~66k MAC operations, int8 kernels at 240 MHz
Cascade decision	<0.01 ms	A handful of comparisons
LED ring update	0.24 ms	8 LEDs at ~30 μ s each
Perceived latency (to LED)	~124 ms	
OLED full-frame refresh	~27 ms	128×64 at 400 kHz
Total to full readout	~151 ms	

The honest observation is that 120 of those 124 milliseconds are the debounce and the post-roll wait, not computation. Signal processing and inference together account for under 6 ms. If the judges weight latency heavily, the post-roll can be cut to 5 samples and the release debounce to 10 ms, bringing perceived latency to roughly 70 ms — at the cost of some settling motion that carries real discriminative content for the flick-terminated spells. That trade is a two-constant change in the shared header and should be made after measuring both, not assumed.

For context, the threshold for perceiving a response as instantaneous is generally placed around 100–150 ms, so the current configuration sits at the edge of that band and the reduced configuration sits comfortably inside it. `LATENCY_MS` is measured on device from confirmed trigger release to decision and rendered on the OLED for every cast, alongside the isolated inference time and the arena high-water mark, as the problem statement requires.

11.2 Memory

Item	Bytes	Notes
Model flatbuffer (flash)	22,520	Measured. 7,174 int8 parameters plus flatbuffer structure and quantisation metadata
TFLM tensor arena (SRAM)	28,672	Allocated. Largest concurrent activation set is ~2 KB; expected use under 10 KB. The real figure is printed on the OLED from <code>arena_used_bytes()</code>
Sample ring buffer	7,680	320×24 bytes (the sample grew with the quaternion)
Capture window	5,520	230×24 bytes
Preprocessing scratch	~20,200	Static, not stack — the FreeRTOS task stack is 8 KB and these buffers exceed it
Input tensor	3,072	768 floats
Framebuffer + LED buffer	1,048	SSD1306 1 KB, ring 24 B
Total SRAM	~66 KB	of 520 KB — about 13%

No heap allocation occurs on the hot path. Every buffer is statically allocated, so the latency figure above contains no allocator variance and cannot fragment over a long demonstration session.

12 On-device processing compliance

The problem statement specifies immediate disqualification if serial communication is used to offload inference. Three independent measures address this:

1. **The demo firmware contains no serial code.** `Serial.begin()` sits behind `#if DEBUG_SERIAL`, and the `infer_nn` build target sets it to 0. The call is not compiled into the binary — verifiable by inspecting the build flags rather than by trusting a promise.
2. **The build flag is a named target, not a hand edit.** It cannot be forgotten, because selecting the demo target is what sets it.
3. **The wand runs on battery.** During the demonstration no USB cable is connected, which makes the point visible to the judges without requiring them to read any code. *If the budget forces the power-bank option in §3.2, this argument weakens and should be replaced by showing the build flags.*

13 Execution plan for the remaining build window

Software is not the bottleneck; the pipeline is built and verified. The critical path is data collection, which five people cannot compress much below its natural duration and which every remaining task depends on. The sensor change adds two bring-up steps that must happen before any collection, and one of them is genuinely dangerous to skip.

When	Task	Owner	Exit criterion
Now	Resolve the budget question with the coordinators	1 person	A written answer on whether a borrowed BNO055 counts against the ₹2,000 cap (§3.1)
Now	Flash <code>pio run -e bringup</code> on a breadboarded ESP32 + BNO055	1 person	Tests 1–3 pass: I2C scan finds <code>0x28</code> , <code>CHIP_ID</code> reads <code>0xA0</code> , <code>begin()</code> returns <code>OK</code> , quaternion norm is unit
+20 min	Bring-up test 4: scale verification	same	Linear acceleration reads ~0 with the wand flat and still — <i>not</i> ~981. A reading near 981 means the chip is handing back raw accelerometer registers and is not fusing. Then a 90° roll integrates to 75–105°.
+35 min	Bring-up test 5: mounting geometry R4	same	Tip pointed at the floor, body-frame "up" reads approximately (−1, 0, 0), confirming +X points at the tip. Then a tip-up flick shows Y dominant and negative, and peak a in the 1.5–4 g band. <i>Under 0.5 g means the lever-arm model does not describe this build and the bootstrap corpus must be regenerated with the geometry you actually have (§4.2).</i>
+50 min	Bring-up test 6: 10-minute clock-stretching stress test	same	Zero I2C errors, max read under 6 ms, with the wand waved and the LED ring lit throughout. <i>Do this before soldering anything permanently</i> — clock stretching on this family produces an intermittent freeze that passes every short test and dies during the demo.
+1.5 h	Trigger button on GPIO27 + 100 nF; flash <code>-e logger</code>	same	A one-second hold produces ~130 rows, every <code>dt_us</code> at 10000 ± 200 , duplicate rate under 12%; <code>collect.py</code> accepts the capture
+2 h	Grip-invariance bring-up test	1 person	Perform the same physical gesture twice with the wand rolled ~90° differently in the hand. The anchored channels should look near-identical while the body-frame channels differ completely. This validates the whole representation in one minute and produces a figure for the report.
+2 h	Parity check against the first real capture	1 person	<code>PARITY OK</code> on real data, not only bootstrap, including zero gate-verdict mismatches
+2.5 h	<code>audit_corpus.py</code> data/raw on the first 50 captures R4	1 person	All classes inside their plausibility bands. <i>If they are not, stop and reconcile before collecting 500 more</i> — the bands are what the divisors, the gate and the arena were sized against, and a mismatch means one of them is describing a different wand (§9.5).
+3 h	Full collection push, all five casters	all 5	≥525 captures in <code>data/raw</code> ; the loader guard stops complaining
parallel	Enclosure per the §3.5 stack-up: BNO055 rigidly bonded at 370 mm from the butt , +X towards the tip	1 person	Sensor cannot move inside the tube; the measured distance is within ±5 mm of 370 and matches <code>SENSOR_FROM_BUTT_M</code> . Photograph the mounting — the report needs it, and it is the first thing to check if a gesture misfires.
parallel	Power: set MT3608 to 4.50 V before connecting anything	1 person	4.50 V measured on an open output; Li-Po polarity verified with a multimeter and marked
+6 h	<code>train_cnn.py</code> on real data, LOCO	1 person	A real accuracy figure with non-zero variance across folds
+7 h	<code>calibrate.py</code> — thresholds, gate percentiles, impact sweep	same	<code>WAND_THRESHOLDS_CALIBRATED 1</code> ; the skipped pytest now runs and passes

+8 h	Flash <code>infer_nn</code> , measure real latency and arena use	1 person	Numbers on the OLED; §11 updated with measurements replacing estimates
+10 h	Adversarial testing — team deliberately trying to cause misfires	all 5	Zero misfires across 100 deliberate Muggle movements
+12 h	Confusion matrix, figures, report update, PPT, demo video	2 people	Submission package assembled

14 Risk register and fallbacks

Risk	Likelihood	Impact	Mitigation
BoM exceeds ₹2,000	High	High	Four levers ranked in §3.2. The first is a five-minute question to the coordinators and should be asked today.
No spare IMU	Medium	Critical	New in R3 and the most serious hardware risk. The MPU-6050 BoM carried two modules at ₹75 each; a spare BNO055 is unaffordable. <i>Mitigation</i> : handle the sensor as the single point of failure it is — solder rather than breadboard for the demo, strain-relieve the wires, and keep the R2 MPU-6050 chain in git history so a dead sensor means reverting a branch and retraining, not redesigning.
I2C clock-stretching lockup	Medium	Critical	Bus at 100 kHz, <code>Wire.setTimeout(1000)</code> , and the 3-minute stress test as a gate before soldering. Duplicate-frame counting gives an early warning in the field.
Gyro calibration not reached before a judge casts	Medium	Medium	Firmware refuses to arm and says so on the OLED. Better to show "CALIBRATING, hold still" for four seconds than to accept a cast the fusion cannot support.
CNN underperforms on real data	Medium	High	<code>infer_dtw</code> build target — banded DTW over k-medoid templates, no training required. One flag, same firmware, same preprocessing chain.
Lumos/Alohomora confusion	Medium	Medium	Anticipated in the design: the top-1/top-2 margin exists for this pair, and calibration breaks ties toward a larger margin. R3 should help materially — the two differ in the <i>direction</i> of the loop, which is now carried explicitly by the quaternion channels rather than having to be inferred from a spinning body frame.
Saturation at ± 4 g on sharp casts	High	Low	Expected, derived, and measured. With a 0.45–0.99 m lever arm the rail is reached by Alohomora on 9.1% of samples and Wingardium Leviosa on 4.8% (§9.5) — not by Stupefy, which was R3's guess. A saturated flick is still unmistakable, the divisor now spans the true ± 5 g reportable range (§6.5), and the impact test separates a knock from a cast by duration rather than magnitude.
Data collection runs short	Medium	High	8× augmentation reduces the required real sample count; the collection script randomises variation so fewer captures still span the space.
Trigger button ruled out	Low	High	<code>AUTO_SEGMENT=1</code> target already builds; Stage 4 temporal persistence is implemented and compiles only in that mode.
Preprocessing divergence	Low	Critical	Ruled out and continuously testable (§9.2), with two verified negative controls.
A synthetic figure reaching the report	Low	High	Three independent locks (§9.4) plus a mandatory stamp on every printed number.

15 Appendix: repository layout and reproduction

wand/ ├─ platformio.ini ├─ shared/ │ └─ wand_config.h │ └─ wand_thresholds.h └─ training/	5 targets: bringup, logger, infer_nn, infer_dtw, infer_nn_autoseg single source of truth – constants AND the wand's geometry AUTO-GENERATED by calibrate.py; the three measured thresholds
--	--

└─ wand_config.py	Python mirror of the headers
└─ preprocess.py	REFERENCE signal chain: quaternion anchoring + the Stage 1 gate
└─ dataset.py	loading, the synthetic guard, LOCO splitter
└─ augment.py	five physically exact transforms + re-anchoring
└─ train_cnn.py	model, LOCO harness, int8 quantisation, C array export
└─ calibrate.py	measures the thresholds and both Stage 1 sweeps; writes the header
└─ diagnose.py	argmax vs cascade, threshold sweep
└─ synth_bootstrap.py	QUARANTINED generator – rigid-body physics from the geometry
└─ audit_corpus.py	physical plausibility audit; works on real corpora too
└─ collect.py	the only program allowed to write data/raw
└─ parity_check.py	Python vs C++ preprocessing AND Stage 1 verdict
└─ test_config_parity.py	header vs Python constant and layout drift
└─ firmware/	
└─ common/	
└─ bno055_fusion.{h,cpp}	THE BNO055 driver – one copy, linked by all three targets
└─ wand_types.h	struct RawSample – one definition, four consumers
└─ wand_tools/bno_bringup.cpp	6-test acceptance suite incl. the mounting-geometry check
└─ wand_logger/wand_logger.cpp	collection firmware, serial ON, emits the CSV contract
└─ wand_infer/	
└─ wand_infer.cpp	demo firmware: sampler, TFLM, cascade, feedback
└─ preprocess.{h,cpp}	line-for-line port of preprocess.py
└─ reject.{h,cpp}	the five-stage cascade
└─ parity_host.cpp	host build of the chain – no Arduino dependency
└─ model_data.{h,cc}	AUTO-GENERATED int8 model as a C array
└─ wand_model.tflite	22,520 bytes
└─ tools/hostcheck/	g++ type-check of all six firmware units, stubbed Arduino + TFLM
└─ docs/	
└─ report.html	this document; code blocks are SYMBOL references, not line ranges
└─ build_report.py	resolves them from the repository at build time
└─ data/	
└─ raw/	real captures ONLY – written by collect.py alone
└─ synthetic/	quarantined; carries DO_NOT_TRAIN_FOR_REPORT.txt

Reproducing every measurement in this report

```
# firmware -- compiles without the ESP32 toolchain
./tools/hostcheck/check.sh

# pipeline, in this order
python training/synth_bootstrap.py --casters 6 --reps 18 --noise-reps 45
python training/audit_corpus.py data/synthetic
python -m pytest training/test_config_parity.py -v
python training/parity_check.py --data ../data/synthetic --n 60      # from training/
python training/train_cnn.py --data data/synthetic --allow-synthetic --epochs 60
python training/calibrate.py --data data/synthetic --allow-synthetic --epochs 60

# report
python docs/build_report.py

# and, once real gestures exist -- in this order:
python training/audit_corpus.py data/raw      # BEFORE collecting the other 475
python training/parity_check.py --data ../data/raw --n 30
python training/train_cnn.py --data data/raw
python training/calibrate.py --data data/raw --misfire-budget 0.0
```

If the wand you build is not the wand this report describes

Three numbers in `shared/wand_config.h` describe the physical object: `WAND_LENGTH_M`, `SENSOR_FROM_TIP_M` and `SENSOR_FROM_BUTT_M`. If the tube ends up a different length, or the sensor lands somewhere else along it, change them there — not in the generator — then regenerate the corpus, re-run the audit, and re-run both Stage 1 sweeps. Everything downstream follows, because nothing downstream re-types those numbers. A wand whose geometry differs from its config file will still train, still run, and still classify; it will simply do so slightly worse, for reasons nothing will report.

Document revision R4 · 1 September 2026. Every figure attributed to a measurement was produced by the commands above and can be reproduced from the repository. No accuracy figure in this document is derived from human gesture data; §1.1 states precisely what has and has not been measured, and §13 gives the plan that closes the gap. BNO055 register addresses, operating-mode values and scale factors are taken from the Bosch Sensortec BNO055 datasheet.